

Underwater Debris Detection Using YOLOV10

Faiza Rehman , Mariam Rehman

PII: S2468-0133(26)00029-X
DOI: <https://doi.org/10.1016/j.joes.2026.02.011>
Reference: JOES 648



To appear in: *Journal of Ocean Engineering and Science*

Received date: 12 December 2024
Revised date: 21 February 2026
Accepted date: 25 February 2026

Please cite this article as: Faiza Rehman , Mariam Rehman , Underwater Debris Detection Using YOLOV10, *Journal of Ocean Engineering and Science* (2026), doi: <https://doi.org/10.1016/j.joes.2026.02.011>

This is a PDF of an article that has undergone enhancements after acceptance, such as the addition of a cover page and metadata, and formatting for readability. This version will undergo additional copyediting, typesetting and review before it is published in its final form. As such, this version is no longer the Accepted Manuscript, but it is not yet the definitive Version of Record; we are providing this early version to give early visibility of the article. Please note that Elsevier's sharing policy for the Published Journal Article applies to this version, see: <https://www.elsevier.com/about/policies-and-standards/sharing#4-published-journal-article>. Please also note that, during the production process, errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

© 2026 THE AUTHORS. Published by Elsevier B.V. on behalf of Shanghai Jiao Tong University.
This is an open access article under the CC BY-NC-ND license
(<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

Highlights

The research fulfills the Sustainability Development Goal (SDG 14) which addresses the challenges related to Life Below Water.

The research work has both theoretical and practical contributions. The theoretical contribution of the study is to address the problem of underwater debris. The practical contribution of the research includes the fine-tuning of YOLOV10s and hyper parameter tuning of YOLOV10s to achieve efficiency and effectiveness.

The major contributions of the study are as follows.

- The proposed method is a novel YOLOV10-based detection approach for underwater debris detection which is potentially more effective and efficient than the existing models of the YOLO series.
- The fine-tuning of YOLOV10s and hyper parameter tuning of YOLOV10s has been performed to increase efficiency and effectiveness.
- The comprehensive comparison of YOLOV10 with the other object detection models i.e. YOLOV8, YOLOV7 and YOLOV5 models has been performed.
- The experiment has been performed to assess the superiority of the proposed model by testing it on two different datasets.

Underwater Debris Detection Using YOLOV10

Faiza Rehman¹, Mariam Rehman^{1, *}

¹Government College University Faisalabad, Pakistan.

Abstract:

Underwater debris detection is an important issue directly affecting marine life and biodiversity. The proposed work introduces a YOLOv10 based detection framework for underwater debris detection. YOLOv10 utilizes a lightweight and efficient network with a backbone, neck, and detection head, where the backbone learns important visual features, the neck improves multi-scale feature fusion with path aggregation, and the detection head outputs object classes and bounding boxes with high accuracy in detection. The proposed model combines optimizer-based hyperparameter adjustment, exploring the impact of optimizers, SGD, Adam, and AdamW, across different learning rates and batch sizes to improve convergence and generalization. The experimental tests were executed on a UW_Garbage_Debris_Dataset of 15 object classes and ICR-19 dataset with the performance measured by Precision, Recall, F1-score, and mean Average Precision (mAP) metrics. The optimized YOLOv10 model, with lower learning rate and lower batch size, trained with the AdamW optimizer produced better detection accuracy and training stability while attaining a precision of 99%. The findings validate that the proposed model balances detection precision and computational efficiency effectively and performs better in comparison with other previous models. Future research will involve building a real-time deployable system through establishing a robust dataset for dynamic underwater environments and integrating the model onto embedded systems for live testing with ROV-based video feeds.

Keywords: Object Detection, Deep Learning, YOLOV10, Debris Detection, Underwater Litter Detection.

1. Introduction

Due to the growth of worldwide industries, the amount of garbage has increased significantly. This type of waste can lead to pollution of air, water, and soil which can be dangerous for human health, degradation of the ecosystem, and reduction in biodiversity. Underwater garbage cleaning, also known as underwater cleanup or marine debris removal involves efforts to remove trash and debris from oceans, seas, rivers, and other water bodies. The process is critical for protecting marine ecosystems, wildlife, and human health process which is crucial for preserving marine ecosystems as well as preserving the beauty and health of oceans and waterways [1]. Currently, 8 million tons of plastic waste enter the ocean annually, harming human health, ecosystems, tourism, economy, and also marine life [2] [3]. Therefore, reducing the detrimental effect can be achieved by investigating the cause of marine litter and controlling it well [3].

There are different deep-learning neural networks, including Mask R-CNN, YOLOv5s, CNN, improved EfficientDet [4], ResNet50-YOLOv3 [5], and YOLOv4 [6], used for debris detection. The existing methods for underwater debris detection are inefficient, rely on multiple algorithms for specific tasks, and still have research gaps for improvement in efficiency and effectiveness. Therefore, there is a need for faster and more effective solutions using advanced algorithms.

The research proposes an object detection approach using YOLOv10 to detect underwater debris. The proposed study uses the YOLOv10s model because YOLOv10s model has outstanding detection performance. The model has a small size and a small number of parameters. YOLOv10 is used because the YOLOv10 model is considered a deep network, which has multiple convolutional layers that work together to detect complex, multi-scale patterns within images, identifying objects with high accuracy and speed. YOLOv10 is used for complex tasks i.e. real-time object detection, and used to recognize objects of varying shapes, sizes, and orientations.

The main objective of the research is to implement an advanced model i.e. YOLOv10s which achieves better efficiency and effectiveness than the existing models. The most recent YOLOv10 version is a feasible choice for real-time underwater debris detection. In this research study, the fine-tuning and optimization of YOLOv10s are performed to find the optimum solution in terms of effectiveness and efficiency.

The major goal of the study is to reduce pollution, propose improved strategies for waste management, clean rivers and oceans to preserve natural ecosystems, and also clean the environment. To improve environmental awareness, the research proposes a deep learning method that uses an enhanced methodology to detect underwater debris. Therefore, the development of a modern detection method could reduce the negative effects that pollution has on marine species and enhance the general sustainability and well-being of marine ecosystems. Therefore, the purpose of the research is to maximize performance on the UW_Garbage_Debris_Dataset and present an effective deep-learning model for underwater debris detection. The research fulfills the Sustainability Development Goal (SDG 14) which addresses the challenges related to Life Below Water.

The research work has both theoretical and practical contributions. The theoretical contribution of the study is to address the problem of underwater debris detection. The practical contribution of the research includes the fine-tuning of YOLOv10s and hyper parameter tuning of YOLOv10s to achieve efficiency and effectiveness.

The major contributions of the study are as follows.

- The proposed method is a novel YOLOv10-based detection approach for underwater debris detection which is potentially more effective and efficient than the existing models of the YOLO series.
- The fine-tuning of YOLOv10s and hyper parameter tuning of YOLOv10s has been performed to increase efficiency and effectiveness.
- The comprehensive comparison of YOLOv10 with the other object detection models i.e. YOLOv8, YOLOv7 and YOLOv5 models has been performed.
- The experiment has been performed to assess the superiority of the proposed model by testing it on two different datasets.

2.1. Related Work

The research study [6] discussed the importance of marine debris detection for protecting the marine ecosystem and highlighted the challenges of accurate detection due to complex underwater environments, small and deformed debris, and potential occlusions. YOLOTrashCan based on YOLOv4 was proposed for marine debris detection. The dataset, sourced from J-EDI deep sea images, includes 1000 video segments from various Japanese sea areas. There are two versions of the dataset i.e. TrashCan-Instance with 22 debris categories and TrashCan-Material with 16 material categories. The dataset comprised 7212 RGB images with 6065/1147 and 6008/1204 images for training and validation on TrashCan-Instance/Material versions respectively. The experiments on both datasets produced a Precision of 0.75/0.74, Recall of 0.62/0.57, and F1 score of 0.66/0.64 for TrashCan-Instance/Material datasets respectively. The YOLOTrashCan network achieved detection accuracies of 65.01% and 58.66% on the two datasets. In comparison with other detection networks i.e. single shot detector (SSD), Efficientdet-D0, YOLOv3, and YOLOv4 gave good performance for YOLOTrashCan on both datasets which provided a lower false detection rate and better object positioning compared to YOLOv4. Future research suggests exploring dilated parallel modules (DPMs) in smaller one-stage object detection algorithms the improvements.

The research work [7] identified limitations in current models and proposed improvements to the YOLOv5s architecture. The dataset containing roughly 11,600 images, was categorized into seven classes i.e. "fishing gear", "cloth", "glass", "metal", "natural debris", "plastic", and "rubber". The dataset was created by collecting images and video frames from the JAMSTEC collection. The study employed data augmentation techniques i.e. mosaic, mixup, and slide stretching to diversify the training data. DSDebrisNet's backbone leveraged the CBS (Convolution + BatchNorm + SiLU) structure, 1x1 convolution kernels, and Spatial Pyramid pooling - fast (SPPF) modules for efficient feature extraction and multi-scale information fusion, crucial for accurate deep-sea debris detection. DSDebrisNet surpassed other methods i.e. ResNet50-Faster R-CNN, ResNet50-SSD, and YOLOv3 in detection accuracy and speed, achieving a mAP0.5 of 92.8 and a mAP0.5:0.95 of 72.4 while running at 60 frames per second (FPS). The experiments on video footage further validate DSDebrisNet's real-time capability and support the development of a sustainable ocean with underwater machines. The proposed method can detect debris even with potential misclassification due to unidentifiable features. Therefore, it does not guarantee perfect results in all cases. Future research will explore utilizing multiple frames to improve detection accuracy within the temporal channel.

The proposed work [8] investigated the effects of marine debris on marine life. The main focus was on using a specific deep learning technique (CNN) to recognize underwater debris and aquatic animals in images. The dataset was obtained from a Japanese agency (JAMSTEC). A dataset of 1644 images was used for training, with an additional 100 images for testing. CNNs were chosen due to their ability to handle large datasets and a specific function (logistic sigmoid) to achieve binary classification (litter or animal). The results showed a 95% confidence interval. The Precision was 85%, the Recall was 94%, and the F1 score was 90% for animal images. The Precision was 0.983, the Recall was 0.84, and the F1 score was 0.88 for litter images. The study highlighted limitations. The biggest limitation was to find publicly available underwater images of debris. The second challenge was linked to the manual labeling of data which was time-consuming. Additionally, many images were not usable due to poor quality, reducing the dataset size and potentially affecting the process of training. The future research will use image segmentation or object detection methods.

The research work [9] focused on using AUVs (Autonomous underwater vehicles) to deal with the growing issues of plastic pollution in the ocean. The paper highlighted a key challenge i.e. current datasets for training debris detection systems are limited and insufficient. The study acknowledged several difficulties in marine debris

detection, including degradation of debris over time in the ocean environment, variations in lighting and water conditions, and the sheer diversity of debris types. These factors make accurate detection a complex task. The paper used synthetic data to address these limitations. They created realistic underwater scenes with simulated debris, aiming to mimic real-world polluted environments. A large existing dataset named CleanSea corpus with categorized marine debris images served as a reference for creating the synthetic data. To train a detection system using both real and synthetic data, a specific neural network architecture called Mask Object-Based Convolutional Neural Network was employed. To further increase the training data, augmentation techniques like flipping, blurring, and color adjustments were used. The results confirmed that the synthetic data improved detection accuracy, and achieved a score of 63.5% for object detection and 65.5% for material identification mAP0.50. The future research directions include exploring advanced data generation techniques like Generative Adversarial Networks (GAN) and investigating methods to improve performance in scenarios with diverse debris types.

The Xue [5] created a fast and accurate method to find trash in the deepsea using deep learning. The dataset used real deep-sea from a big database (JAMSTEC) that has 7 types of trash. 85% dataset was used to train a ResNet50-YOLOV3. The algorithm was fast and good at finding trash at the highest accuracy. The future work was to improve the program to find even more types of trash to avoid mistakes, and eventually use it on videos too.

The algorithm studied [4] tackled the growing problem of plastic pollution in oceans and the major focus on improving autonomous underwater vehicle (AUV) vision for the detection of debris. The EfficientDet model improved version (D0) achieved 1.5% higher accuracy than existing models. The study involved creating a publicly available dataset (WPBB) containing images specifically focusing on plastic bags and bottles in water environments. The dataset included 900 fully labeled images. The model trained was for comparison, the original EfficientDet versions (D0-D3) and their variations on two other datasets i.e. Trash-ICRA19 and TrashCan. Trash-ICRA19 contained 1200 samples and was used to train the improved EfficientDet model, achieving 47.4% Average Precision (AP) with D0 and 49.6% AP with D3. TrashCan was a larger dataset with 16 categories and 6008 labeled images. Additionally, training EfficientDet on the plastic-specific WPBB dataset resulted in a lower accuracy of around 15% AP, in comparison with TrashCan. Moreover, both accuracies were significantly lower in comparison with the achieved results on Trash-ICRA19. The improved EfficientDet model demonstrated promising performance when trained on the more general debris datasets i.e. Trash-ICRA19 and TrashCan, achieved an AP between 40-50% and 10-20% approximately. The future work will implement image segmentation techniques to gain more detailed information about object shapes beyond the bounding boxes and test the debris detection model in real-world ocean environments.

The research work [10] concerned with the growing threat of human-made garbage in the ocean and its impact on marine life. The study focused on using deep learning to detect litter on the sea floor. There are several challenges in detecting seafloor litter i.e. camera angles, lighting variations, different types of litter, degradation of litter over time, and background clutter like rocks or seaweed. The dataset consisted of a video frame containing 635 images from an underwater camera towed in Ermoupolis Bay, Greece. The dataset included various types of litter across a large seafloor area. The original images were quite large; the images were resized for efficiency also augmented. Deep learning offers various object detection and segmentation techniques. The research used Mask R-CNN, which involves two CNNs a backbone CNN for feature extraction and a head CNN for various tasks like region proposal, classification, bounding box prediction, and mask prediction (to capture the exact shape of the litter). The network was trained on the seafloor litter dataset with 11 litter classes, its performance was monitored through training and

validation losses. Transfer learning with fine-tuning was employed to leverage pre-trained knowledge from a different image recognition task. The model achieved 62% mean average Precision (mAP) on the testing set, demonstrating good performance and higher precision compared to other methods. The future work can explore alternative network architectures and data pre-processing techniques to improve the accuracy of underwater garbage detection. The research tackles the growing problem of plastic pollution in the ocean, focusing on how it harms the environment. Plastic waste in the ocean has become a major concern, and current management strategies are failing to keep up. The plastic pollution disrupts the delicate balance of ocean ecosystems.

The proposed research work [11] applied object detection techniques to underwater images to identify marine litter. The existing methods struggle with a lot of challenges i.e. objects looking similar to each other i.e. interclass similarity and objects appearing differently depending on the situation i.e. intra-class variability. The research addressed the issues by using a dataset called Trash-can, containing over 7,000 underwater images collected from another dataset (J-EDI).

The dataset was divided for training and testing, with some images used to develop the detection system. To increase the size of dataset the data augmentation techniques were also used and improve the system's ability to handle unseen data. The research presented a deep learning model called MLDet, designed to detect underwater garbage. MLDet was based on an existing method, RetinaNet but incorporated additional features to handle the complexities of underwater images. Additionally, MLDet used a special type of convolutional neural network i.e. deformable convolutional networks to better manage the irregular shapes of marine debris objects. The MLDet achieved a mean Average Precision (mAP) of 42.3%, and performed well in comparison with other object detection models i.e. Faster R-CNN and Cascade R-CNN.

MLDet was more accurate at identifying marine debris in underwater images. The research addressed a few limitations. Underwater light scattering and refraction can blur and distort images, leading to misdetections by MLDet. Additionally, other background objects like seaweed, rocks, or strangely shaped natural elements can be mistaken for litter, causing false positive errors. The future work should address the remaining challenges caused by light and background clutter.

The algorithm explained in [12] focused on finding trash in the ocean to help monitor and protect marine ecosystems. It's difficult to see objects underwater from far away using cameras; therefore, sonar was needed for deep-sea exploration. Additionally, sonar images have speckles noise that makes them blurry and hard to analyze for tasks like finding and performing object segmentation in the image. The regular ways to get rid of speckles in sonar images involve processing techniques that focus on the image itself and spatial domain filtering. Additionally, these methods can be unreliable. The research proposed a new way to remove speckles that don't need clean images to start with. The approach aims to work well even in different situations i.e. high generalization capability. Another challenge for finding trash in sonar images is separating objects from the background images. The study used two datasets i.e. ARACATI 2017 and MDFD containing 12 categories of objects/trash.

The new method improved the clarity of the image's denoising effect by 4.2% and also improved how well it finds trash detection accuracy. Future research on processing sonar images has two main areas of focus while bridging the gap between methods that use labels for the entire image and methods that need labels for each tiny part of the image. Developing new techniques to handle the unique properties of sonar images, which often lack in terms of fine details and textures needed for precise object recognition.

The research work [13] highlighted the growing issue of marine debris and its negative impact on the environment. It emphasizes the importance of underwater target detection technology for ocean garbage collection and ecological protection. The study proposed an improved YOLOv5 model trained on a specific dataset i.e. trash_ICRA19. The model performed very well with an Accuracy of 87.2%, Recall of 97%, and mean Average Precision (mAP) of 90.6% for underwater object detection. Additionally, it boasts a fast detection speed of 0.29 seconds per image. The improved YOLOv5 model performed very well in comparison with other detection models i.e. YOLOv2, Tiny-YOLO, Faster R-CNN, and single shot detector (SSD) especially in plastic waste recognition, achieved an impressive 95.1% mAP for plastic objects.

The research work [14] introduced a novel underwater litter detection model built upon YOLOv7-tiny. This model tackles the complexities of similar-looking litter types and variations within a single class. By refining the YOLOv7 small model using a marine litter dataset, the researchers achieved a significant improvement, reaching a mAP of 81.8%. This surpasses the performance of other object detection methods. The future work aims to enhance the model's versatility and detection abilities, as well as expanding the underwater trash dataset. The proposed work [15] presented a new underwater trash detection model named YOLOv8-C2f-Faster-EMA. This model refined the YOLOv8 baseline by bolstering the backbone network, incorporating multi-scale calibrated detection, and enhancing the C2f module. The model was trained on the Trash-ICRA19 dataset, the model demonstrated exceptional accuracy and speed, particularly in detecting small underwater objects. By comparing various YOLOv8 variants, the study found that YOLOv8-C2f-Faster-EMAv3 achieved the highest mAP (84.6%) and fastest processing time (3.042 hours). The future research could focus on addressing issues like sample imbalance, data quality, close proximity detection, and improving the model's ability to generalize to new shapes.

2.1. Object Detection

Object detection can be performed using single-stage and two-stage methods. Single stage performs object detection in a single pass but two-stage perform detection in two-stages. Therefore, the single-stage models are faster than the two stage models [16], [17]. Table 1 shows the comparison of single-stage models and two-stage models.

Table 1. The comparison of single stage and two-stage methods.

Features	Two Stage	Single Stage
Process	Two phases: Proposal generation and classification. [18]. (Diwan, Anirudh, and Tembhumne 2023).	Single pass: Combined localization and classification. [18]. (Diwan, Anirudh, and Tembhumne 2023).
Accuracy	Higher (Sirisha et al. 2023)	Nearer to the two-stage detectors (Diwan, Anirudh, and Tembhumne 2023).
Speed	Slower (Diwan, Anirudh, and Tembhumne 2023).	Faster (real-time possible) (Diwan, Anirudh, and Tembhumne 2023).

Computation	High resource demands (Diwan, Anirudh, and Tembhurne 2023).	Low resource demands (Diwan, Anirudh, and Tembhurne 2023).
Cost	More expensive (to run) (Diwan, Anirudh, and Tembhurne 2023).	Less expensive (to run) (Diwan, Anirudh, and Tembhurne 2023).
Object Size	Detects small and large objects well (Diwan, Anirudh, and Tembhurne 2023).	Better for large objects And struggles with small objects (Diwan, Anirudh, and Tembhurne 2023).
Network Architecture	Complex, heavyweight backbone (Diwan, Anirudh, and Tembhurne 2023).	Simpler, lightweight backbone (Diwan, Anirudh, and Tembhurne 2023).
Region Proposals	Generated by separate algorithm e.g., Selective Search (Diwan, Anirudh, and Tembhurne 2023).	Generated within the network (Diwan, Anirudh, and Tembhurne 2023).
Example Models	R-CNN, Fast R-CNN, Faster R-CNN, YOLO, RetinaNet, EfficientDet (Diwan, Anirudh, and Tembhurne 2023).	YOLO , RetinaNet, EfficientDet (Diwan, Anirudh, and Tembhurne 2023).

Table 2. The comparative Analysis of YOLOV1 to YOLOV8.

Features	YOLOv1	YOLOv2/9000	YOLOv3	YOLOv4	YOLOv5	YOLOv6	YOLOv7	YOLOv8
Authors	Joseph Redmon [19]	Joseph Redmon [19]	Joseph Redmon [19]	Alexey Bochkovskiy [19]	Glenn Jocher [19]	Meituan Tech Team [19]	-	Ultralytics [19]
Release Year	2015 [19]	2016 [19]	2018[19]	April 2020 [19]	Soon after YOLOv4 [19]	June 2022 [19]	After YOLOv6 [19]	Jan 2023 [19]
Backbone	Darknet-53 [19] [20]	Darknet-19 (v2)/ Darknet-53 (v2/9000) [19] [20]	Darknet-53 [19] [20]	CSPDarknet-53 [20]	CSPDarknet-53 [19].	CSPStackRepblock [19]	RepConv 3-Stacked ELAN CSPDarknet [19] [18]	Modified CSPDarknet-53 [17]
Architecture	Fully convolutional [19]	Fully convolutional [19]	Feature pyramid network (FPN) [20]	CSP (cross-stage partial) [19]	CSP (cross-stage partial) [19]	RepPAN [18] [19]	Compound scaling [19]	Feature pyramid network (FPN) [19]
Limitations	Low Recall Localization Error (Hussain, 2023), Small Objects Closely Packed Objects [21],	Small Objects [21]	-	-	needs a lot of data for good results [22]	Speed, Accuracy Trade-off [19]	Speed for Edge Deployment [19]	-
Performance	Faster than DPM-v5, Lower mAP than Faster R-CNN [19]	Improved over SOTA detectors[19]	Multi-scale detection [19] [21]	Faster, More Accurate than previous [21]	Improved Speed, Accuracy[21]	Best Speed & Accuracy [19]	Improved Accuracy in comparison with YOLOv6 [19]	Hardware-efficient, Outperforms YOLOv5/6 [19]

Improvements	Grid Division [19] confidence loss [21]	Batch Normalization, Better Positioning, Recall [21] [19] Diwan et al., 2023)	Multi-scale detection, Upgraded Backbone [19] Diwan et al., 2023)	Faster, Lightweight, Self-Adversarial Training [21] [19] Diwan et al., 2023)	Model size flexibility, Faster Training/Inference (Hussain, 2023)	Anchor-free approach, Faster Speed (Hussain, 2023)	Model Scaling (Hussain, 2023)	Focuses on Edge Deployment [22] (Hussain, 2023)
Dataset	PASCAL VOC [19] [17]	PASCAL VOC [17]	MSCOCO [17]	MSCOCO [17]	MSCOCO [19]	MSCOCO [19]	MS COCO [19]	MSCOCO

Table 2 explains that YOLO has consistently upgraded its backbone architecture, progressing from Darknet-53 to modified CSPDarknet-53 for better feature extraction. Throughout the versions, there's a constant push to improve speed and accuracy. YOLO addressed limitations like small object detection and positioning errors over time. YOLOv3's multi-scale detection and YOLOv7's design changes are examples. Therefore, YOLOv4 offered significant advancements. The YOLOv5 seems more focused on refinement. YOLOv6, YOLOv7, and YOLOv8 show a growing focus on real-time detection and edge device deployment, indicating the importance of practicality. YOLOv8, however, appears to be innovative in terms of hardware efficiency. YOLOv9 and YOLOv10 are advanced models which are released in 2024. There was no paper on YOLOv9 and YOLOv10. Overall, the YOLO series demonstrates continuous improvement in object detection, balancing speed, accuracy, and practicality for diverse applications.

2.2. YOLO Variants

There are some variants of the YOLO series from YOLOv5. YOLOv5 comes with different variants concerning computational parameters. YOLOv5s, YOLOv5m, YOLOv5l and YOLOv5x are the variants of YOLOv5 algorithm. The number of parameters and FLOPs are different according to their size. Table 3 shows variants of YOLOv5 with a number of parameters [19], [17].

Table 3. Variants of YOLOv5 with number of parameters.

Model	Parameters
YOLOv5s	7.5 M
YOLOv5m	21.8 M
YOLOv5l	47.8 M
YOLOv5x	86.7M

YOLO-v6 comes in different variants offering a trade-off between accuracy (mAP) and speed. YOLO-v6-N is the fastest variant but has the lowest mAP. YOLO-v6-L-ReLU achieves the highest mAP but is the slowest. (YOLO-v6-small, medium, large), the models with different sizes and parameters. Table 4 Shows variants of YOLOv6 with several parameters [19].

Table 4. Variants of YOLOV6 with number of parameters.

Model	Parameters
YOLO-v6-N	4.3
YOLO-v6-T	15
YOLO-v6-RepOpt	17.2
YOLO-v6-S	17.2
YOLO-v6-M	34.3
YOLO-v6-L-ReLU	58.5

Variants of YOLO-V7 are presented in Table 5. The tiny version YOLO-v7-tiny is less accurate (mAP) than the powerful one YOLO-v7-D6, but it also needs less processing power [19], [17].

Table 5. Variants of YOLOV7 with a number of parameters.

Model	Parameters
YOLO-v7-tiny	6.2
YOLO-v7	36.9
YOLO-v7-X	71.3
YOLO-v7-E6	97.2
YOLO-v7-D6	154.7

Table 6 shows the variants of YOLOV8 and shows the number of parameters of YOLOV8. YOLOV8 offers five variants with different sizes i.e. YOLOv8n, YOLOv8s, YOLOv8m, YOLOv8l, and YOLOv8x. YOLOV8 method performs multiple tasks in a single pass i.e. object detection, segmentation, pose estimation, tracking, and classification [19], [17].

Table 6. The variants of the YOLOV8 model.

Model	Parameters
YOLOV8n	3.2M
YOLOV8s	11.2M
YOLOV8m	25.9M
YOLOV8l	43.7M
YOLOV8x	68.2M

Each version offers models with varying sizes and complexities. More parameters generally indicate higher accuracy but also require more processing power. Table 7 Shows the features-wise properties of YOLO Series.

Table 7.Features wise properties of YOLO Series.

Features	YOLO V1	YOLO V2	YOLO V3	YOLO V4	YOLO V5	YOLO V6	YOLO V7	YOLO V8
Model Architecture [17]	Single Network	Single Network	Darknet-53 + Detection Heads	Single Network	Single Network	Single Stage	ELAN	Modified SPP-YOLO
Input Resolution [17]	448x448	416x416	Configurable (up to 608x608)	Configurable (up to 608x608)	640x640	Configurable (up to 640x640)	Configurable (up to 1280x1280)	Configurable (608x608)
Bounding Box Prediction [17]	With Class Probabilities	With Class Probabilities & Anchor Boxes	With Class Probabilities & Objectness Score	With Class Probabilities & Confidence Scores	With Class Probabilities & Confidence Score	With Class Probabilities & Confidence Score	With Class Probabilities & Confidence Score	With Class Probabilities
Object Classes [17]	20	80	80	80	Customizable depending on dataset	80	80	80
Activation Function [17]	Leaky ReLU	Leaky ReLU	Leaky ReLU	Mish	SiLU (Swish)	SiLU	Leaky ReLU	Leaky ReLU

In this paper, a YOLOv10s-based approach for underwater debris detection is selected. The proposed research has been performed to address the limitations of the previous work and to provide improved accuracy and efficiency with advanced models, i.e. YOLOV10s. In addition, the study performed a comparison using different object detection i.e. YOLOV8, YOLOV7, and YOLOV5 models. Additionally, the proposed model was tested on the different datasets to check the performance of the proposed model.

There are five main sections of the paper: Section 1 consists of an introduction and literature review on deep learning approaches, debris detection strategies, underwater debris detection, and the YOLO series. Section 2 contains an effective deep-learning model for underwater debris detection also discussed the training process and the dataset used. The primary findings are described in Section 3, which also presents the outcomes of the experiments. The research's synthesis is given in section 4. Section 5 describes the conclusion.

3. Materials and Methods

3.1. Dataset

The experiments used the UW_Garbage_Debris_Dataset for the training of YOLOV10. The dataset contains 5130 underwater debris images classified into 15 litter categories i.e. "Mask," "can," "cellphone," "electronics," "bottle," "glove," "metal," "misc," "net," "pbag," "pbottle," "plastic," "rod," "sunglasses," and "tire". The dataset was collected from kaggle.

Table 8 shows the dataset distribution of images in training, testing, and validation sets to train and test the YOLOV10 model.

The UW_Garbage_Debris_Dataset was selected for the proposed research due to its high resolution and clear image quality.

Table 8. Shows dataset details.

Dataset	
Training	3628
Testing	501
Validation	1001

The dataset was divided into training, testing, and validation sets. The training set consists of seventy percent 70% of the images, the validation set consists of 20%, and the remaining 10% for the test set. The pre-trained YOLOV10 algorithm was trained on 640x640 input images. The training was run at 30 epochs. The performance of the YOLOV10 model was evaluated using metrics of batch size, learning rate epoch size, and optimizer. Table 9 shows the optimized configuration of training parameters for the YOLOV10 model. Table 10 shows the pre-trained parameters of YOLOV10.

Table 9. Optimized configuration of training parameters for the YOLOV10 model.

YOLO Models	Optimizers	Batch Sizes	Learning Rates	Epoch Sizes
YOLOV10s	Adam	16	0.001	30
YOLOV10s	Adam	32	0.01	30
YOLOV10s	Adam W	16	0.001	30
YOLOV10s	AdamW	32	0.01	30
YOLOV10s	SGD	16	0.001	30
YOLOV10s	SGD	32	0.01	30

Table
Pre-

Trained parameters of YOLOV10.

Pre-Trained Model	Optimizers	Batch Sizes	Learning Rates	Epoch Sizes
YOLOV10s	AdamW	32	0.000526	30

3.2. Experimental Setup

The runtime environment for the YOLOV10s model consisted of an NVIDIA GPU, and python as the deep learning framework. Several experiments were conducted to determine the optimal number of training epochs. The training process was conducted using the python programming language within kaggle and colab environments.

The hardware and software specifications used for training, validating, and testing of YOLOv10 model are provided in Table 11.

Table 11. Hardware and software specifications for YOLOV10.

Components	Details
RAM	8GB
OS	Windows OS
Platform	Colab notebook
GPU	NVIDIA
Model	YOLOV10

3.3. Model Architecture

YOLO network consists of 23 layers, including convolutional layers, up-sampling layers, concatenation layers, and custom modules from the ultralytics library. In convolutional neural networks, the key function of convolutional layers is to identify and extract important features from the input images.

Concatenation layers combine feature maps from different parts of the network which includes operations related to object detection, potentially including residual connections or spatial attention mechanisms. The network follows a typical encoder-decoder architecture. The encoder part (layers 0-10) reduces the spatial resolution of the feature maps while increasing the number of channels (feature depth) to capture complex features. The decoder part (layers 11-22) increases the spatial resolution back using up sampling and combines features from different depths using concatenation. The final layer (layer 23) is a custom v10Detect module from the ultralytics library, responsible for generating object detection outputs like bounding boxes and class probabilities.

YOLO algorithms offer a primary model for real-time object detection because of their remarkable balance between detection performance and computational complexity. The YOLOv10 architecture reflects advanced and highly optimized structure designed for real-time object detection. It shows the key elements backbone, neck, and head each playing a unique and crucial role that lends itself to the model's high accuracy and performance.

3.3.1. Backbone

The backbone network acts as the basis of YOLOv10 and is tasked with extracting meaningful visual features from input images. YOLOv10 uses an improved CSPNet that involves multiple cross-stage connections to guarantee better feature representation and effective reuse of features across layers.

This structure allows the model to retain dense spatial and semantic information, enhancing overall detection accuracy and efficiency. Moreover, the backbone utilizes a Spatial Pyramid Pooling Fast (SPPF) layer to extract spatial features at different scales, allowing the model to efficiently detect objects of different sizes and distances [26],[27].

3.3.2. Neck

The neck network, which is positioned between the head and backbone, is crucial for feature fusion and upsampling. YOLOv10 employs a Path Aggregation Network (PAN) architecture that combines multi-scale feature maps via upsampling operations.

This integration process is what makes sure that high-level semantic attributes are optimally integrated with low-level spatial information, resulting in more detailed and robust feature representations. This makes the model scale better and adapt to object detection in various environments and scales [26],[27].

3.3.3. Head

At the very top of the architecture is the head network, which produces the final detection results. YOLOv10 presents a decoupled head design, deviating from the conventional anchor-based detection approach employed by previous YOLO models.

Alternatively, it takes an anchor-free strategy, enabling the model to make object locations and categories predictions directly, with lower computational overhead and easier training. The training is also boosted by the addition of Distributional Focal Loss (DFL), which enhances the learning process through enhanced localization accuracy and confidence estimation.

Overall, YOLOv10 represents a balanced combination of accuracy, efficiency, and speed. It supports NMS-free training by virtue of consistent dual assignments from its networks, which heavily decreases inference latency while keeping high detection performance. Additionally, with its lightweight yet responsive backbone, feature-intensive neck, and optimized head, YOLOv10 is one of the most highly optimized and strongest real-time object detection frameworks available, providing outstanding results on a vast range of applications.

YOLO models utilize Non-Maximum Suppression (NMS) at the post-processing step, adding more inference latency. Additionally, various elements in the YOLO architecture add computational overhead, constraining the overall efficiency and potential of the model. In tackling these issues, the YOLOv10 algorithm adds architectural modifications that further optimize both performance and speed. Particularly, YOLOv10 employs dual assignments for NMS-free training, which essentially does away with the necessity of NMS without compromising detection accuracy and also lowering inference latency. The configuration of uniform dual assignments for NMS-free training is shown in Figure 1, where a uniform matching metric m is applied. This metric can be given as follows:

$$m(\alpha, \beta) = s \cdot p^\alpha \cdot \text{IOU}(\hat{a}, a)^\beta$$

(1)

where $\hat{a}$ and a denote the instance and prediction bounding box, p is the score of classification, s indicates the spatial prior, and α and β are two hyperparameters to balance the contributions of the location regression task and the semantic prediction task [26],[27].

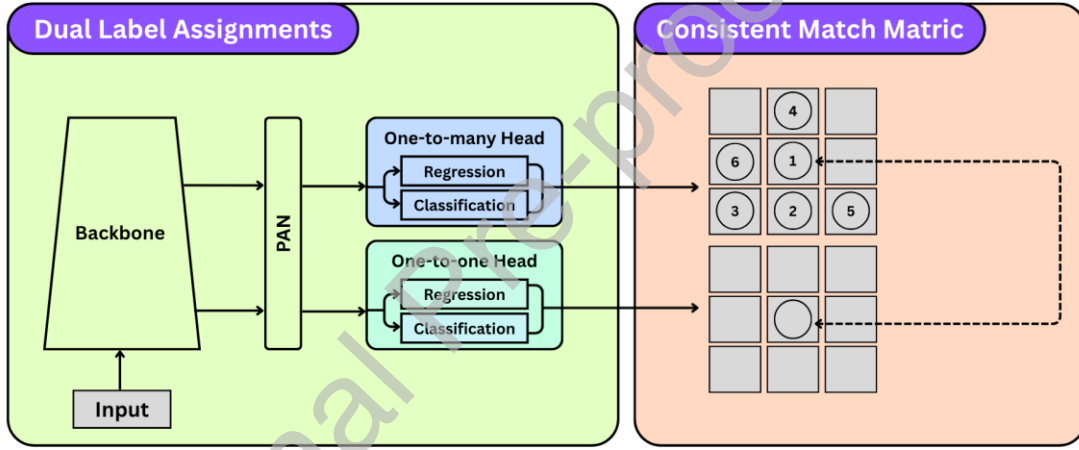


Figure.1. YOLOv10 Architecture.

3.4. Computational Complexity

The computational complexity of YOLOv10 depends on its architecture and operations, including convolutions and activation functions. There are multiple parameters which influence complexity i.e. the number of parameters, FLOPs, and the overall layer structure.

To calculate the Big O complexity, need to analyze the core operations: convolutions, grid cells, and bounding box predictions. The complexity is determined by the number of operations per image input and how this scales with input size, grid size, and model depth.

3.4.1. Convolutional Layer Complexity

Convolutions are the most computationally intensive part of YOLO models. The complexity of a convolutional layer is given by:

$$O(k^2 * C_{in} * C_{out} * W * H)$$

Where:

- k^2 : kernel size (e.g., 3x3 or 5x5)
- C_{in} : number of input channels
- C_{out} : number of output channels
- $W * H$: the spatial size of the input image

YOLOv10 employs optimizations like spatial-channel decoupled down sampling and depth wise separable convolutions. These techniques reduce complexity by breaking down convolutions into pointwise and depth wise operations, leading to lower computational demands compared to standard convolutions.

3.4.2. Grid Cell and Bounding Box Predictions

YOLOv10 divides the input image into an $S \times S$ grid of cells. Each cell predicts B bounding boxes and C class probabilities. The complexity of the process is given by:

$$O(S^2 * B * C)$$

Where:

- S^2 : number of grid cells (proportional to input image resolution)
- B : number of bounding boxes per cell (usually 3)
- C : number of classes

The prediction process is repeated for each input image, contributing substantially to the overall complexity.

3.4.3. Non-Max Suppression (NMS)

After generating predictions, Non-Max Suppression (NMS) is used to eliminate overlapping bounding boxes. The complexity of NMS scales with the square of the number of predicted boxes, expressed as:

$$O(N^2)$$

Where N is the number of predicted boxes. YOLOv10 incorporates optimizations like partial self-attention (PSA) and efficient block designs to reduce the computational cost of these post-processing steps.

Combining these components, the total complexity of YOLOv10 for inference can be approximated as:

$$O(S^2 * B * C) + O(\text{Convolutions}) + O(N^2)$$

Convolutional layers are the primary contributors to complexity, especially with high-resolution images and deeper architectures. However, YOLOv10's innovations like large-kernel depth wise convolutions and rank-guided block design enhance efficiency without substantially increasing complexity.

3.5. Proposed Methods

The dataset images were divided into training, testing and validation sets. The YOLOV10 model was trained, tested, and validated using the same platform in Google Collaboratory. The hyper parameters tuning and optimization of YOLOv10s was performed. The training and testing of the model was performed using NVIDIA. Figure 2 details the research methodology, including the model training process, performance evaluation after hyper-parameters tuning of YOLOv10, and the specific hyper parameters chosen. Figure 2 also illustrates the comparisons made during this process.

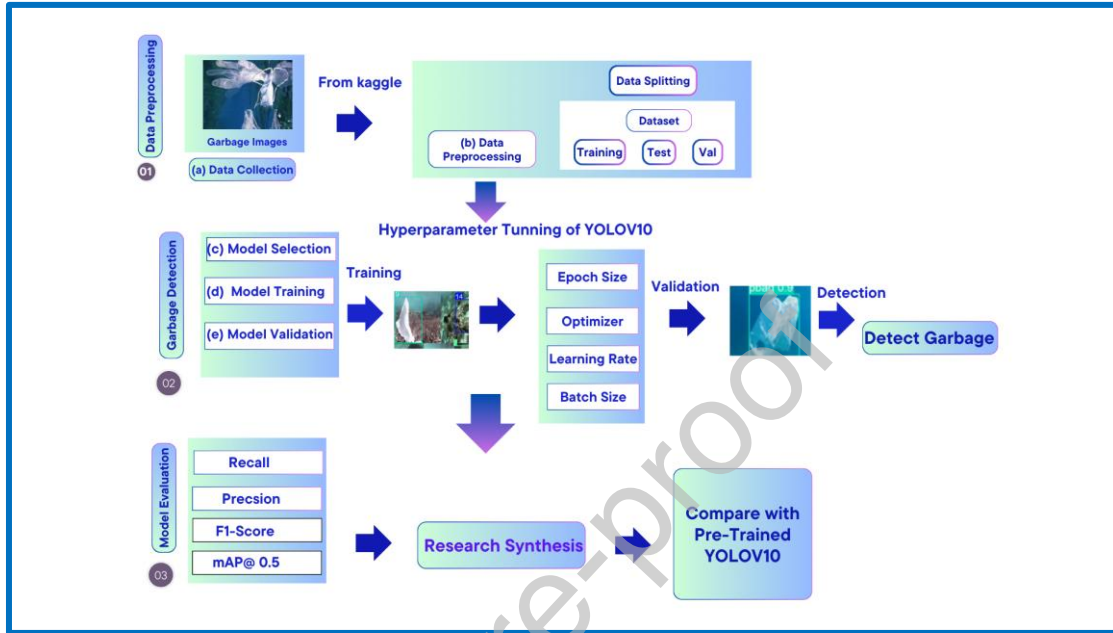


Figure 2. Research Methodology.

The proposed study investigated the impact of the pre-trained YOLOv10 model and improved YOLOv10 performing hyper parameter tuning and optimization. The performance and training speed of the model varies significantly depending on hyper parameter tuning and the type of optimizer [23].

First, the hyper parameters have been set. The study selected three optimizers i.e. stochastic gradient descent (SGD), Adam, and AdamW for optimization. Two batch sizes were selected i.e. 16 and 32 with two learning rates i.e. 0.001 and 0.01 respectively.

The proposed study compared the performance of the pre-trained YOLOV10 model against the optimized YOLOV10 model which was trained with three different optimizers i.e. Adam, AdamW, and SGD, two different learning rates, and two different batch sizes.

The low learning rate with low batch size gives good performance. The high learning rates often resulted in poor model performance with small batch sizes. Conversely, lower learning rates tended to yield better performance when using small batch sizes [23].

The learning rate and batch size are correlated. Therefore, the learning rate of 0.001 with a batch size of 16 and 0.01 learning rate with the batch size of 32 has been set to perform experiments. The lower learning rate of 0.001 and lower batch size of 16 has been chosen. The higher learning rate was 0.01 and the higher batch size 32 has been chosen.

The proposed study selected these learning rates and batch sizes to optimize model performance, a careful balance between learning rate and batch size is a crucial step. Lower learning rates, i.e. 0.001, are generally more stable and can prevent divergence, especially when using smaller batch sizes i.e. 16. This combination allows for more precise gradient updates and better convergence. Conversely, higher learning rates of 0.01 can accelerate training, however, they require larger batch sizes, i.e. 32, to maintain stability and avoid overfitting to come up with optimal solutions. By selecting these specific parameter settings, the study aims to strike a balance between efficient and effective training and accurate model performance.

Table 10 shows the Pre-Trained parameters of YOLOV10. The batch size means a number of images were processed at a time before updating the model's internal parameters. All the experiments run on the same epoch size i.e. 30.

The pre-trained YOLOV10 used AdamW optimizer, 0.000526 learning rate, 0.0005 decay with 30 epoch size. AdamW is a variation of Adam, a popular optimizer for training neural networks.

Moreover, both are good for large datasets, AdamW tackles weight decay differently, which can improve how well the model learns. Original Adam (Adaptive Moment Estimation algorithm) is efficient for big problems requiring less memory, working faster also demands less training rate compared to SGD, but weight decay might not be ideal. AdamW improves weight decay handling, allowing for better fine-tuning of learning rates. Additionally, Adam and AdamW are the variants of the Adam optimizer [24].

SGD is faster and uses less memory compared to Adam and AdamW. Therefore, SGD doesn't track any information about past gradients for each parameter. Adam and AdamW, on the other hand, require storing and processing the average and variation of past gradients for each parameter, making them computationally more expensive [25]. The AdamW optimizer has high speed, high convergence stability, and Adaptive Learning Rates.

To assess the impact of selected learning rates and batch sizes on model performance, the study explored different combinations of all parameters with 3 different optimizers i.e. AdamW, Adam, and SGD.

The equation for optimizers are as follows.

1: Stochastic Gradient Descent (SGD)

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t) \quad (2)$$

- θ_t : model parameters at step t
- η : Learning Rate
- $\nabla_{\theta} \mathcal{L}(\theta_t)$: Gradient of loss w.r.t parameters.

2. Adam Optimizer

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \\ \theta_{t+1} &= \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \end{aligned} \quad (3)$$

where

- $g_t \nabla_{\theta} \mathcal{L}(\theta_t)$ (gradient)
- β_1, β_2 : exponential decay rates
- ϵ : small constant

3. AdamW Optimizer

$$\begin{aligned}
 m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\
 v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\
 \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}
 \end{aligned} \tag{4}$$

λ : regularization term

3.7. Evaluation Settings and Performance Metrics

The effectiveness of YOLOV10 algorithm in object detection depends on Precision, Recall, F1-score, and Mean Average Precision (mAP). These metrics evaluate the ability of the algorithm, to accurately pinpoint and classify objects within an image. The Precision-Recall Curve (P-R Curve) shows the Precision and Recall for a model. The Precision represents the proportion of positive predictions on the y-axis [28], [29]. The following equation was used to calculate Precision.

$$Precision = \frac{TP}{TP + FP} \tag{5}$$

The following equation was used to calculate the mAP:

$$mAP = \frac{1}{n} \sum_{k=1}^n AP_k \tag{6}$$

The following equation was used to calculate the F1-Score [29].

$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{7}$$

The Recall represents the proportion of true positives identified by the model also called sensitivity [30], [28]. The following equation was used to calculate the Recall.

$$Recall = \frac{TP}{TP + FN} \tag{8}$$

4. Results and Discussions

The study investigated the effectiveness of three optimizers on the YOLOV10s model for underwater debris detection. The study compared the impact of large batch sizes with high learning rates against small batch sizes with low learning rates, within the context of the three optimizers.

According to Table 12, the Adam optimizer achieved a Precision confidence value of 98% and a Recall of 89%, and mAP is 58% when using a learning rate of 0.001 and a batch size of 16. Moreover, when the learning rate was increased to 0.01 and the batch size was doubled to 32, the Precision confidence value dropped to 97% and the Recall significantly decreased to 70%, which also reduce the mAP value to 29%. The study performed experiments using a 30 epoch size because a 30 epoch size gives maximum results and different experiments have been performed to reach an optimum epoch size which gives outstanding results. Moreover, when epoch size increased from 30 the Precision value has been decreased.

Table 12. Results with Adam optimizer.

Precision	Recall	mAP	Accuracy Training	Time	Optimizers	Batch sizes	Learning Rates
97	70	29	48	0.972 h	Adam	32	0.01
98	89	58	58	1.048 h	Adam	16	0.001

Table 12 shows that a smaller batch size of 16 with a lower learning rate of 0.001 leads to better performance in terms of Precision 98%, Recall 89%, and mAP 58% compared to a larger batch size of 32 with a higher learning rate 0.01. Additionally, the training time was slightly longer 1.048 h for the smaller batch size compared to the larger batch size 0.972 h. The experiment showed that using a smaller batch size and a lower learning rate improved the accuracy of the Adam optimizer for underwater debris detection using the YOLOV10s model but at the cost of slightly increased training time.

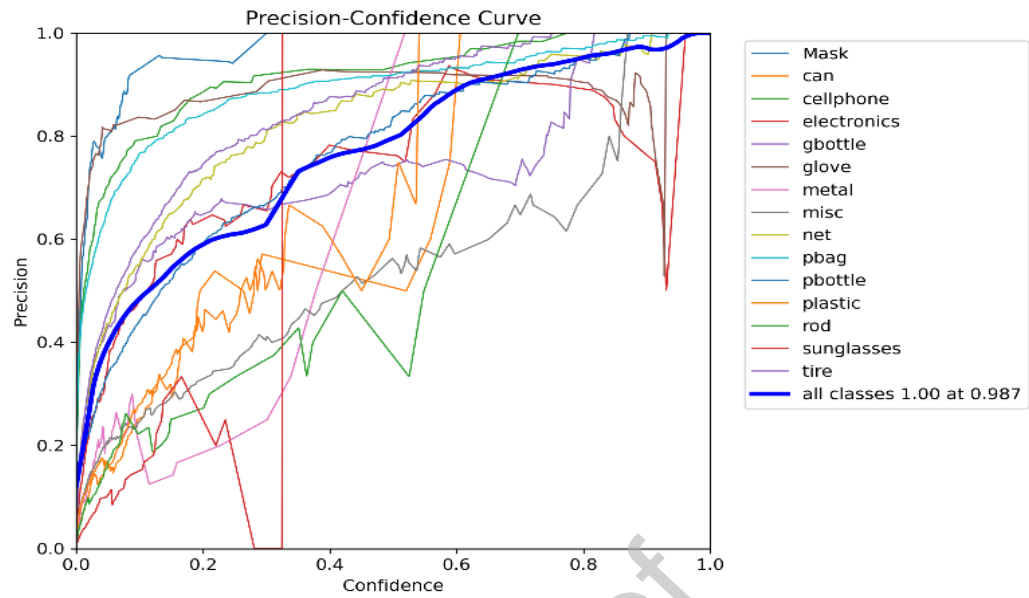


Figure 3. Precision Confidence Curve of Adam with 16 batch size and 0.001 learning rate.

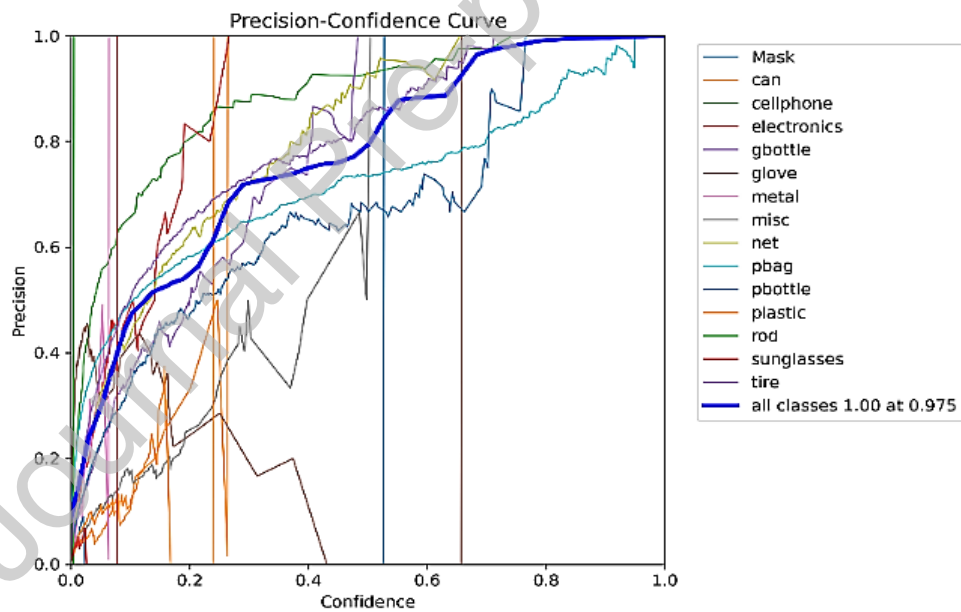


Figure 4. Precision Confidence Curve of Adam with 32 batch 0.01 learning rate.

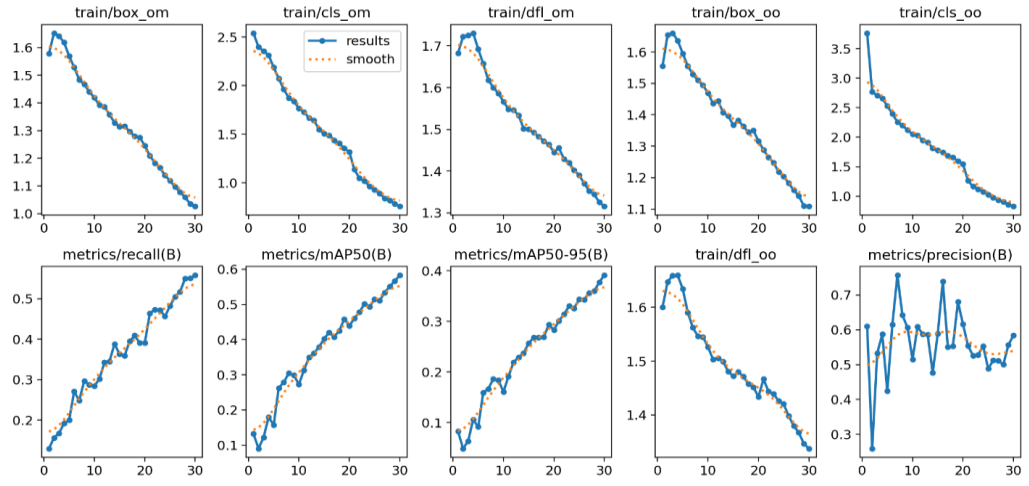


Figure.5. Results of Adam with 32 batch size and 0.01 learning rate.

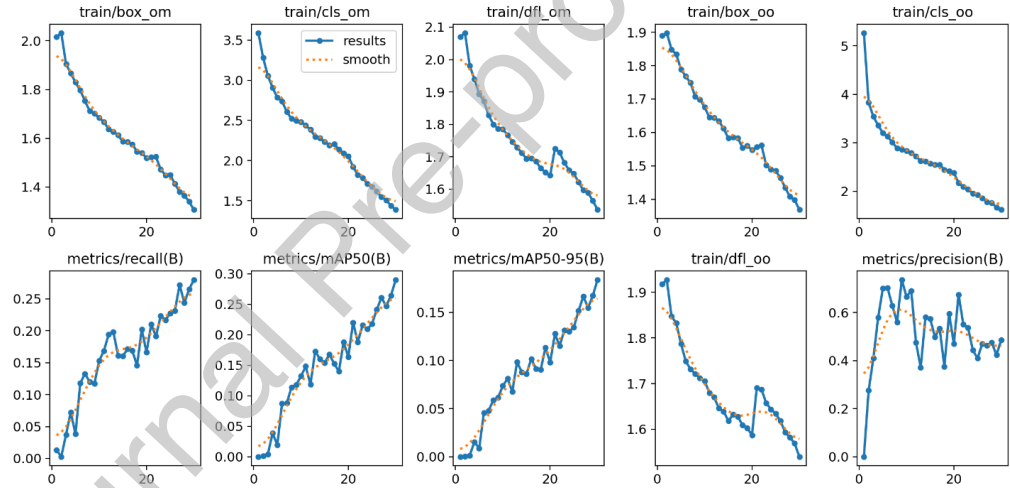


Figure.6. Results of Adam with 16 batch size and 0.001 learning rate.

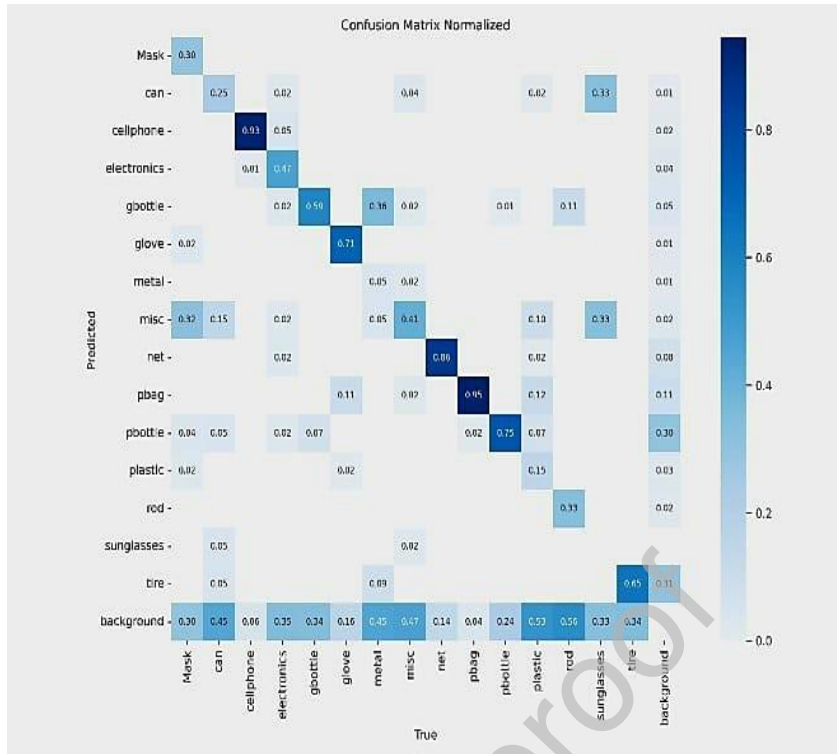


Figure 7. Confusion Matrix of Adam with 16 batch size and 0.001 learning rate.



Figure 8. Confusion Matrix of Adam with 32 batch size and 0.01 learning rate.

Table 13 summarizes the performance of the AdamW optimizer on a YOLOV10s model for underwater debris detection using different learning rates and batch sizes. Table 13 shows that a smaller batch size of 16 with a lower learning rate of 0.001 leads to better performance in terms of Precision 99%, Recall 88%, and mAP 59% compared to a larger batch size of 32 with a higher learning rate of 0.01. Moreover, the training time was significantly faster at 0.012 h for the smaller batch size compared to the larger batch size at 1.005h. Additionally, training time was significantly faster 0.012 hours with the smaller batch size compared to the larger batch size with 0.950h. The training time was

significantly faster with the smaller batch size and lower learning rate. According to Table 12, the AdamW optimizer achieved a Precision confidence value of 95% and a Recall of 88% when using a learning rate of 0.01 and a batch size of 32. Moreover, AdamW with lower learning rate and lower batch size achieved higher Precision and mAP than Adam optimizer with lower batch size and learning rate.

Table 13. Results with AdamW optimizer.

Precision	Recall	Map	Accuracy Training	Time	Optimizers	Batch sizes	Learning Rate
95	0.78 at 0.00	0.39	0.618	0.950h	AdamW	32	0.01
99	0.88 at 0.00	0.59	0.601	0.012h	AdamW	16	0.001

The experiment suggested that using a smaller batch size and a lower learning rate might improve the accuracy of the AdamW optimizer for underwater debris detection with the YOLOV10s model, but at the cost of significantly reduced training time.

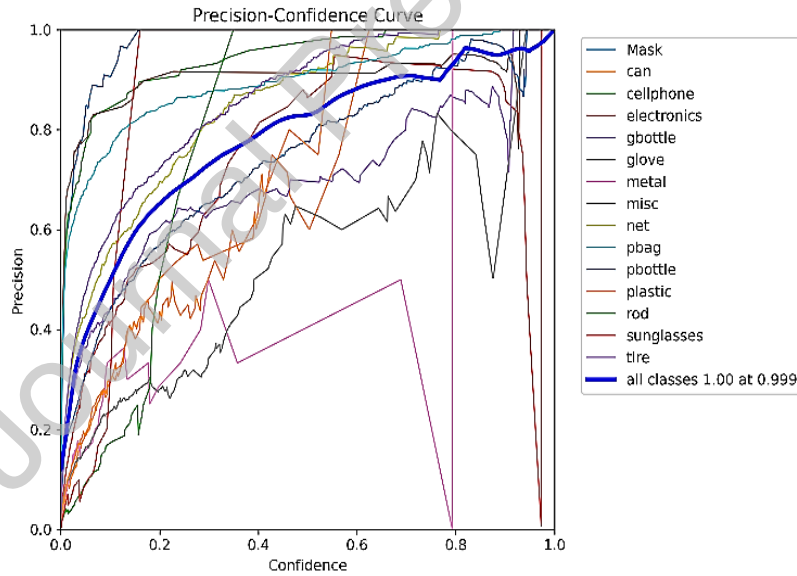


Figure 9. Precision confidence curve of AdamW with 16 batch size and 0.001 learning rate.

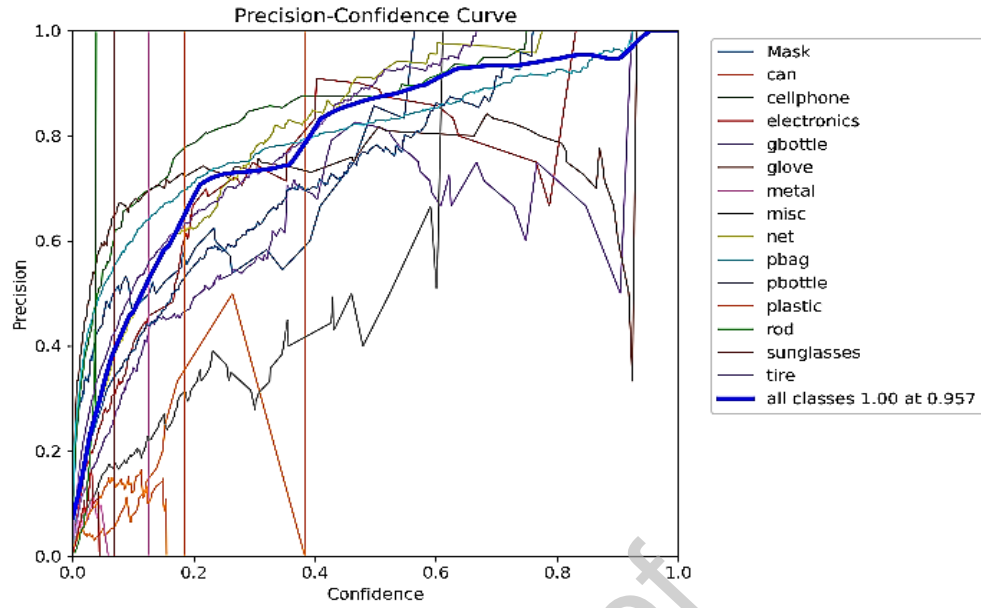


Figure 10. Precision confidence curve of AdamW with 32 batch size and 0.01 learning rate.

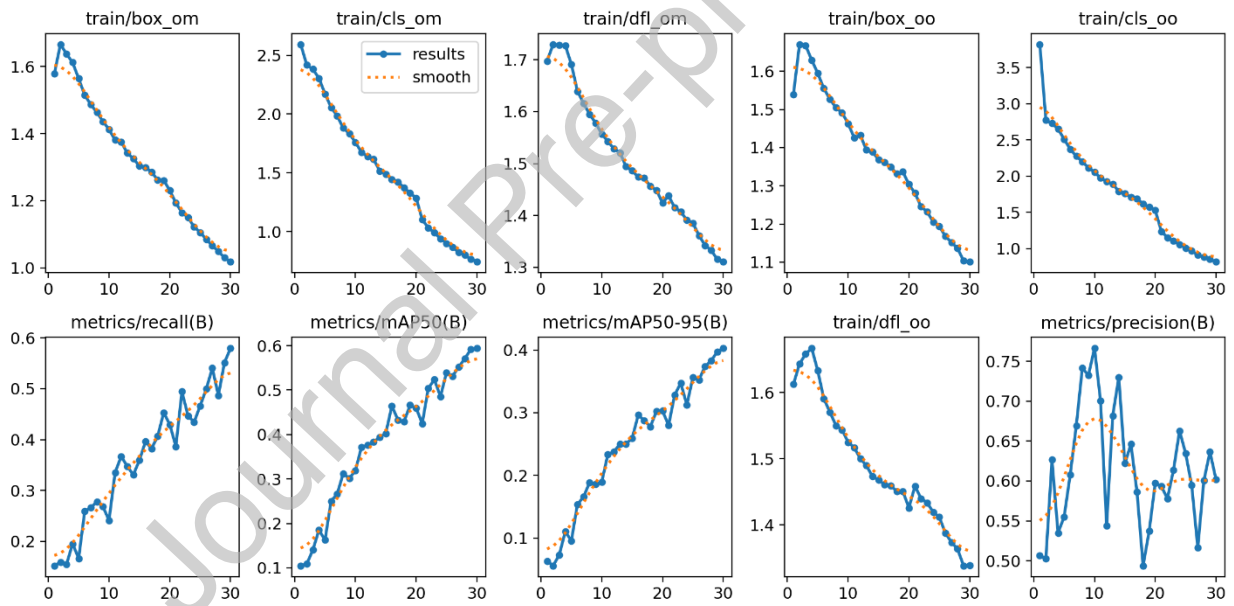


Figure 11. Results of AdamW with 16 batch size and 0.001 learning rate.

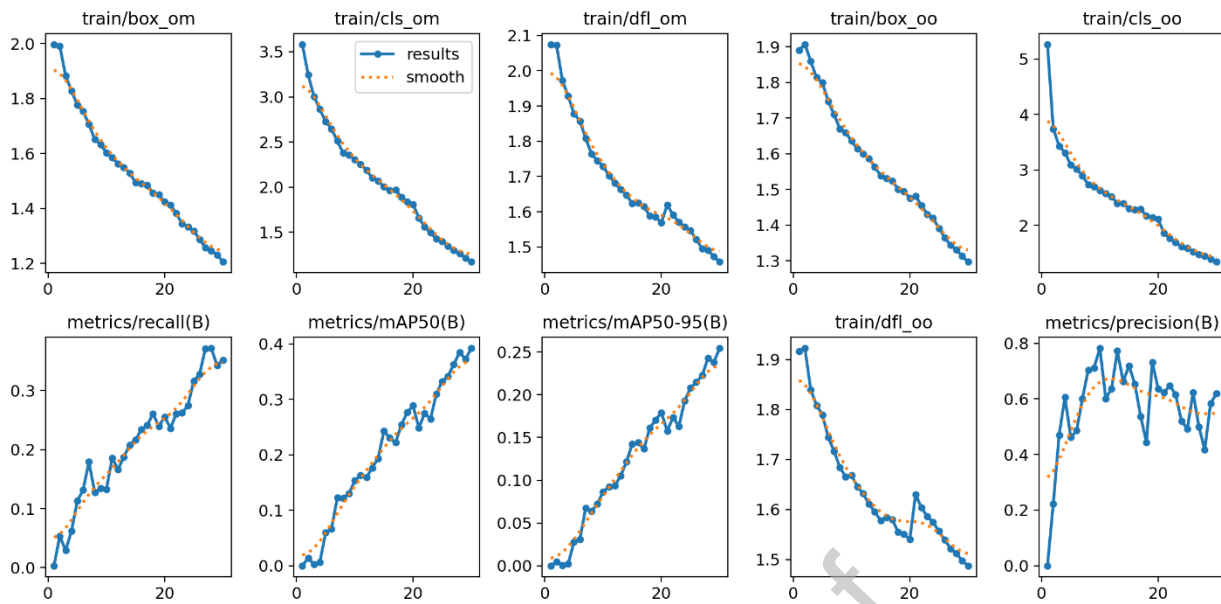


Figure 12. Results of AdamW with 32 batch size and 0.01 learning rate.

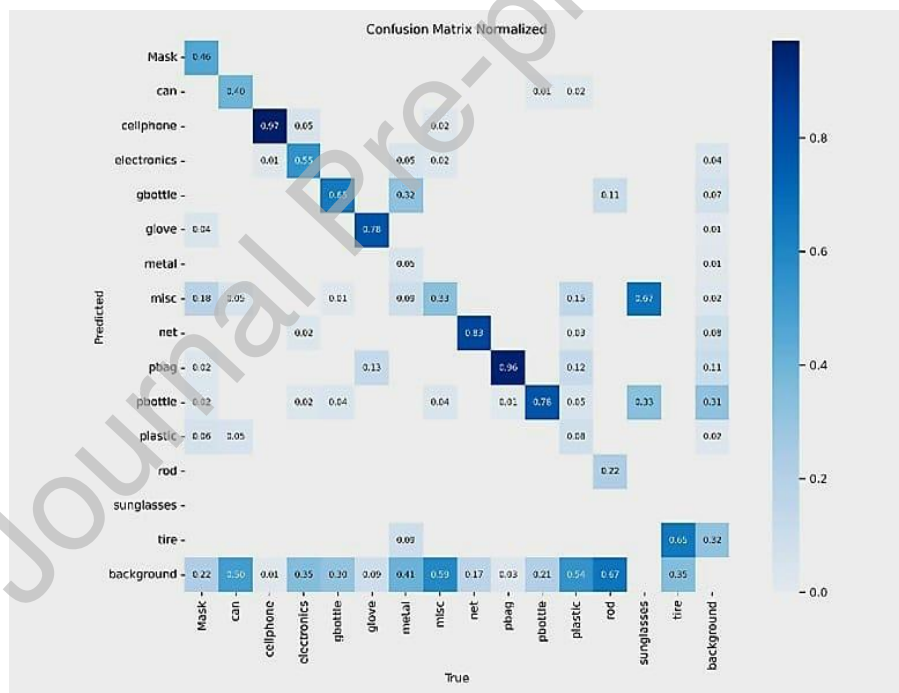


Figure 13. Confusion Matrix of AdamW with 16 batch size and 0.001 learning rate.

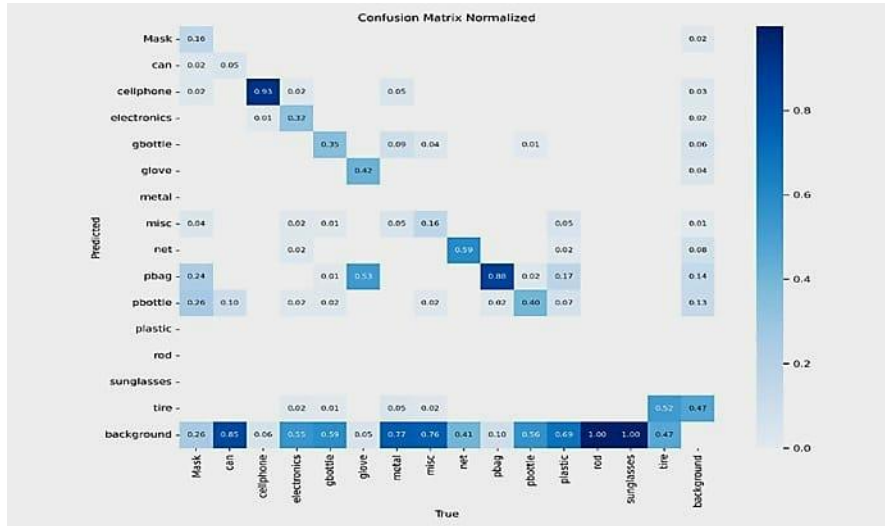


Figure 4. Confusion Matrix of AdamW with 32 batch size and 0.01 learning rate.

Table 14 describes the performance of the SGD optimizer on a YOLOV10s model for underwater debris detection. At a learning rate of 0.001 and a batch size of 16, the model achieved a Precision confidence value of 99% , a Recall of 92% , and an mAP of 74% which is a higher value than all results. The SGD optimizer performed better with a lower learning rate of 0.001 and smaller batch size of 16 in terms of Precision, Recall, mAP, and overall accuracy, with a slightly shorter training time in comparison with using a higher learning rate of 0.01 and larger batch size 32.

Table 14. Results with SGD optimizer.

Precision	Recall	mAP	Accuracy	Time	Optimizers	Batch sizes	Learning Rates
98	0.90 at 0.00	0.71	0.664	0.992h	SGD	32	0.01
99	0.92 at 0.00	0.74	0.789	0.977h	SGD	16	0.001

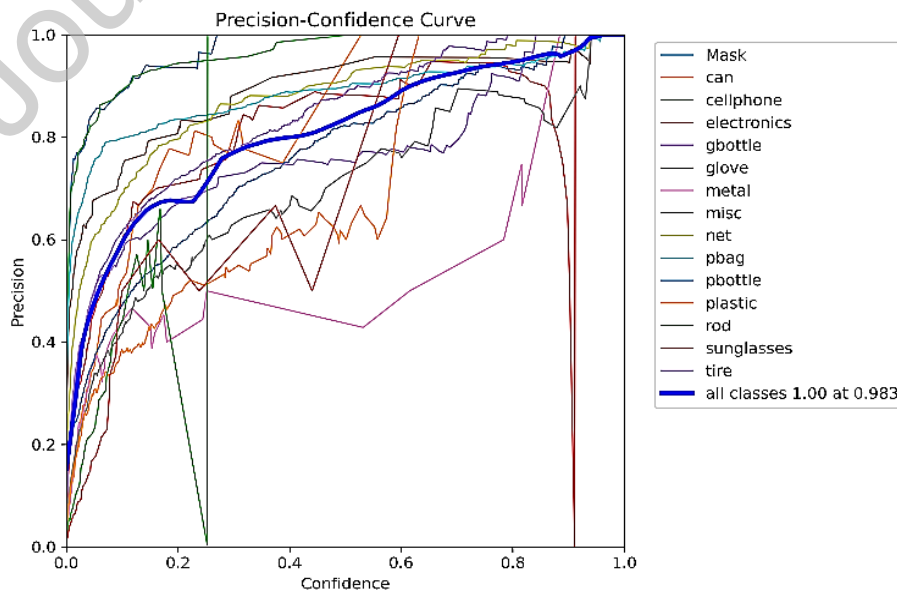


Figure 15. Precision confidence curve of SGD with 32 batch size and 0.01 learning rate.

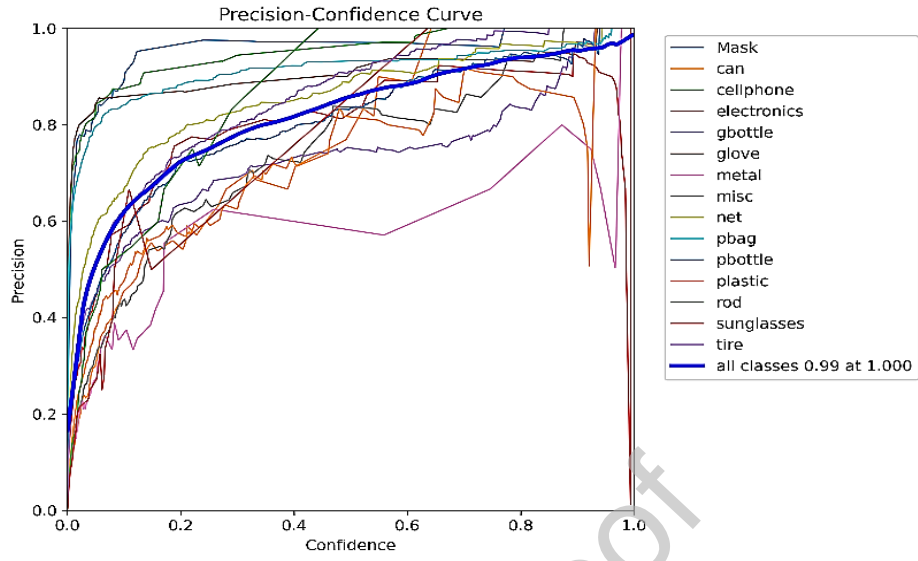


Figure 16. Precision confidence curve of SGD with 16 batch size and 0.001 learning rate.

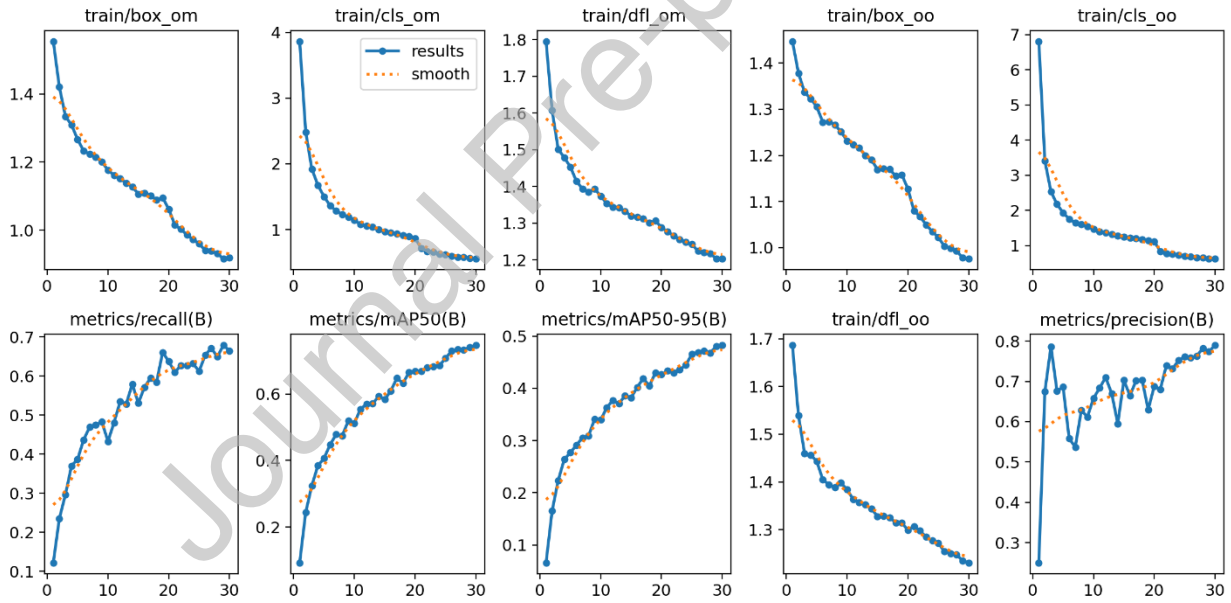


Figure 17. Results of SGD with 16 batch size and 0.001 learning rate.

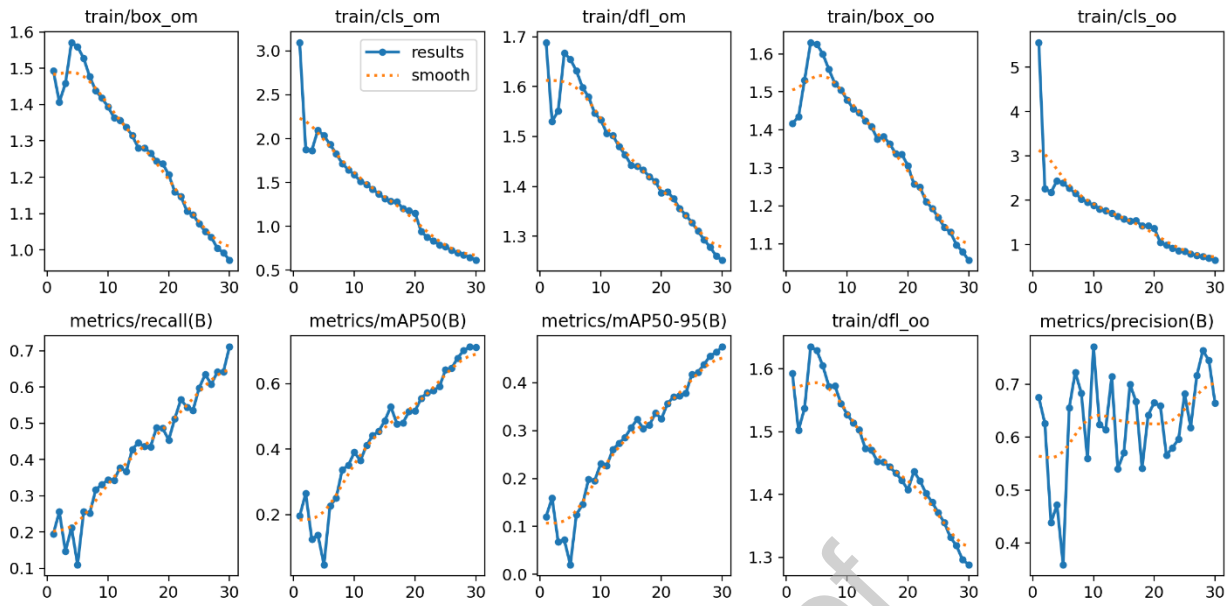


Figure 18. Results of SGD with 32 batch size and 0.01 learning rate.



Figure 19. Confusion Matrix of SGD with 16 batch size and 0.001 learning rate.

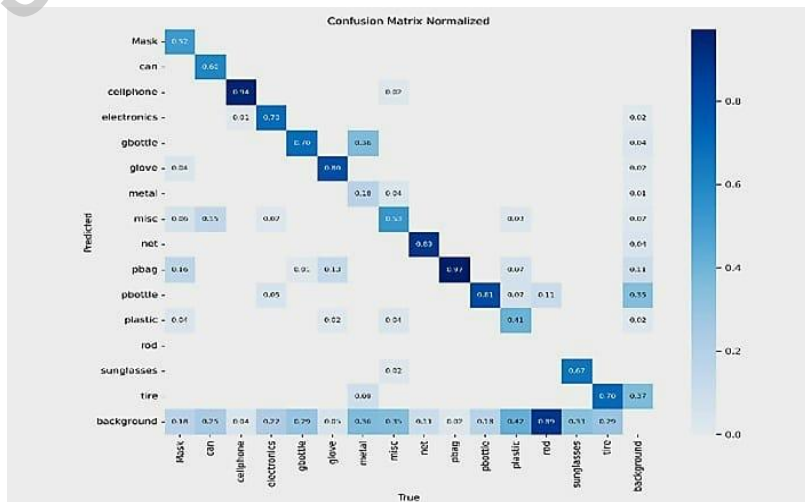


Figure 20 . Confusion Matrix of SGD with 32 batch size and 0.01 learning rate.

The study compared the effectiveness of three optimizers i.e. Adam, AdamW, and SGD on a YOLOV10s model for underwater debris detection. The main findings on the impact of learning rate and batch size were the smaller batch size 16 with a lower learning rate of 0.001 generally leads to better performance in terms of Precision, Recall, and mAP for all three optimizers. SGD, Adam, and AdamW optimizers were most effective with a 0.001 learning rate and a batch size of 16. The AdamW optimizer also performed well with a 0.001 learning rate and a batch size of 16, particularly in terms of Precision and training time. Additionally, AdamW with a 0.001 learning rate and a batch size of 16 was notably faster.

The conclusion was that using a smaller batch size of 16 and a lower learning rate of 0.001 appeared to be the best configuration for all three optimizers in terms of achieving higher Precision, Recall, and mAP for underwater debris detection with the YOLOV10s model.

The authors [23] suggested that there isn't a single best optimizer that works universally across different models. Additionally, the study observed that when using lower learning rates, smaller batch sizes achieved better results [23]. The reference [31] showed that a learning rate of 0.001 achieved optimal training performance. The reference [24] achieved the most promising outcomes when using the AdamW optimizer.

In a comparison of training times for the Adam optimizer, using a batch size of 32 and a learning rate of 0.01 resulted in faster training completion (0.972 hours) compared to a batch size of 16 and a learning rate of 0.001 (1.048 hours). This suggests that for this specific scenario, a larger batch size with a slightly higher learning rate leads to improved efficiency. When comparing training times for the AdamW optimizer, a smaller batch size of 16 and a lower learning rate of 0.001 achieved the fastest training completion (0.012 hours) compared to a larger batch size of 32 and a slightly higher learning rate of 0.01 (0.950 hours). The experiment suggests that for this specific case, using a smaller batch size with a carefully chosen learning rate can significantly improve training efficiency.

Moreover, when comparing training times for the SGD optimizer, a batch size of 32 and a learning rate of 0.01 achieved a completion time of 0.992 hours. Moreover, a smaller batch size of 16 with a lower learning rate of 0.001 resulted in a slightly faster training time of 0.977 hours. Additionally, both settings produce competitive results, the smaller batch size with a lower learning rate shows a higher efficiency for SGD. Overall AdamW optimizer emerges as the most efficient, surpassing other configurations. Additionally, SGD optimizer demonstrates outstanding performance in Recall and mean Average Precision (mAP) compared to other settings. However, both SGD and AdamW achieve similar levels of Precision.

4.1. Comparative Experiment with Different Object Detection Models

To validate the performance of the proposed model, the research compared well-known object detection algorithms such as YOLOV8, YOLOV7, and YOLOV5. The UW_Garbage_Debris_Dataset served as the experimental dataset and the evaluation metrics included Precision, Recall, mAP, and F1 Score. To demonstrate the effective and efficient detection performance of the proposed model, the experiment has been performed to compare.

Table 15. Comparative Results of object Detection Models with Proposed Model.

Model	No of Epochs	Precision	Recall	F1-Score	mAP
YOLOV8	30	96%	89%	73%	77.4%
YOLOV7	30	95.4%	92%	50%	56%

YOLOV5	30	96.6%	78%	36%	41%
Proposed Model	30	99%	92%	71%	74%

The experimental results show that the YOLOV8s have a Precision of 96%, Recall of 89%, mAP of 77.4%, and 68% F1-score with 30 epoch size. The proposed model achieved 99% Precision which is higher than the YOLOV8s model.

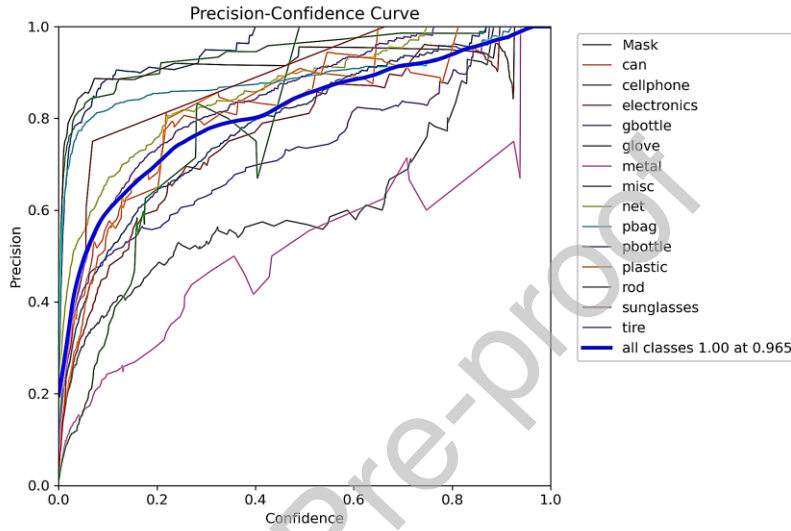


Figure 21. Precision Confidence Curve of YOLOV8s with Proposed Model.

Table 15 shows that the YOLOV7 achieved 95.4% Precision, 92% Recall, 51% F1-Score, and 56% mAP. The proposed model achieves superior Precision in comparison with all the models. Moreover, the proposed model achieved a little bit less Recall rate than the YOLOV7 but the proposed model achieved higher mAP than the YOLOV7 model.

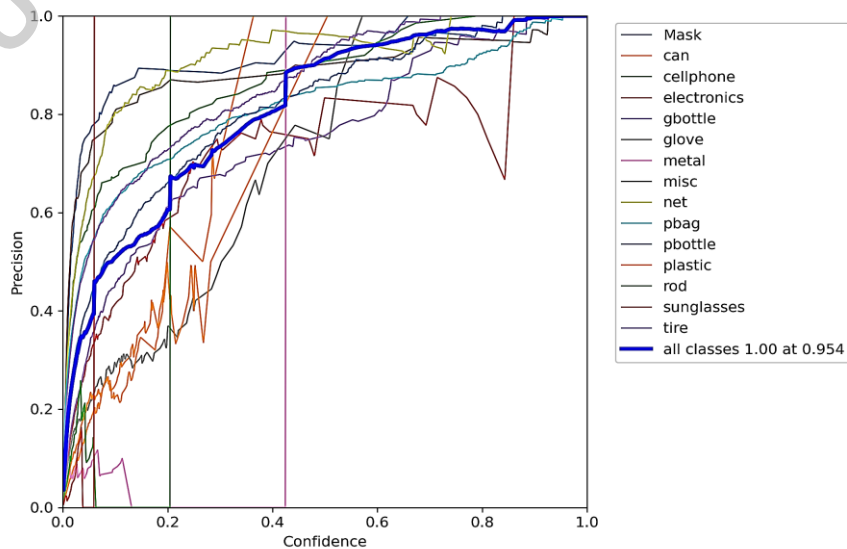
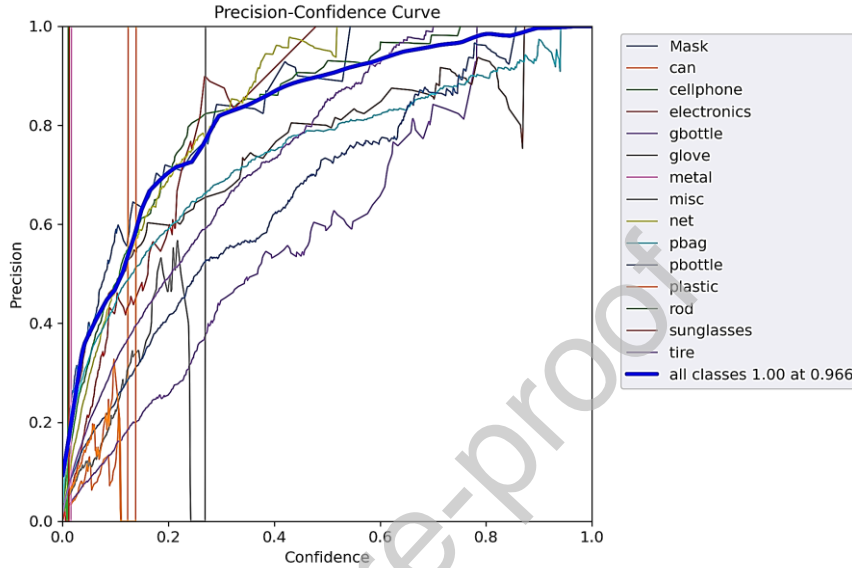


Figure 22. Precision Confidence Curve of the YOLOV7 with Proposed Model.

Table 15 demonstrates the results of the YOLOV5 model with all the results of object detection models. The YOLOV5 model achieved a Precision of 96.6% which is much higher than the YOLOV7 model. Moreover, the YOLOV5 model achieved a Recall of 78%, F1-Score of 36%, and 41% mAP. The proposed model outperformed than the YOLOV5 model. The Proposed model archives outstanding efficiency and effectiveness in comparison with all the models.

**Figure 23.** Precision Confidence Curve of YOLOV5 with Proposed Model.

4.2. Comparison with ICRA19 Dataset

Table 16. Comparison between ICRA19 Dataset and UW_Garbage_Debris_Dataset.

Datasets	No of Epochs	Precision	Recall	mAP
ICRA19 Dataset	30	90%	68%	41%
UW_Garbage_Debris_Dataset	30	99%	92%	74%

Table 16 presents the results of the proposed YOLOV10s model with ICRA19 Dataset and UW_Garbage_Debris_Dataset. The research tested the improved YOLOV10s on two datasets i.e. ICRA19 and UW_Garbage_Debris_Dataset. The ICRA19 dataset achieved a Precision of 90%, Recall of 68% with 41% mAP. The UW_Garbage_Debris_Dataset shows the best results with 99% Precision, 92% Recall and 74% mAP using SGD optimizer.

The proposed model using UW_Garbage_Debris_Dataset outperforms in comparison with overall results. The proposed research selected the UW_Garbage_Debris_Dataset because the resolution of UW_Garbage_Debris_Dataset is high and all the images are clear.

The ICRA dataset consists of just three classes and the resolution of images is not high. The dataset does not clearly define all the classes of debris. Moreover, the UW_Garbage_Debris_Dataset consists of 15 classes. Additionally, the

quality of the training dataset can significantly impact the results of the model. In addition, the UW_Garbage_Debris_Dataset gives the best results in comparison with the ICRA19 dataset.

The quality of the training dataset is a crucial factor influencing model performance, and the UW_Garbage_Debris_Dataset proves superior to the ICRA19 dataset.

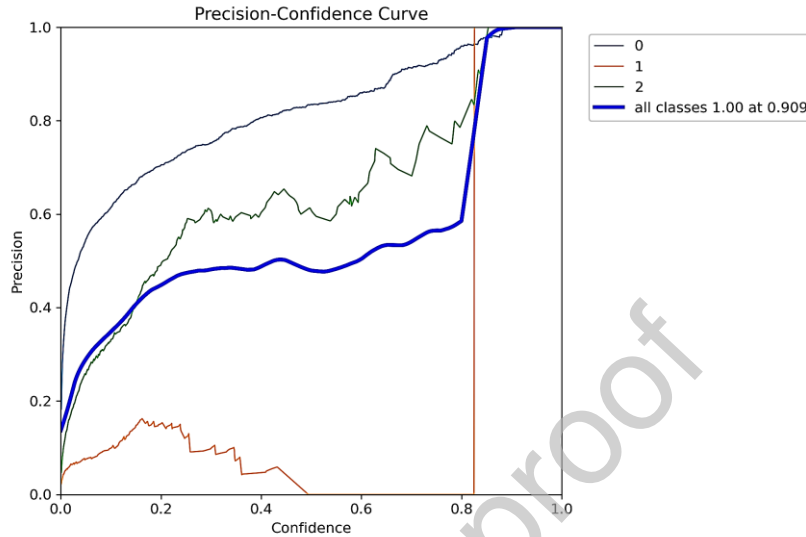


Figure 24. Detection Results with ICRA19 Dataset.

5. Research synthesis

The research synthesis has been shown in Table 17.

Table 17. Research Synthesis with Existing Models.

Proposed by	Model Used	Dataset Used	Results
Politikos et al [10]	Mask R-CNN.	635 video frames	62%
Cheng et al [12]	Modified U-Net structure	- MDFD - ARACATI 2017	8.63 ENL and 0.96 SSI on MDFD 0.55 ENL and 0.91 SSI on ARACATI
Ma et al [11]	Retina Net	Trash Can	42.3%
Teng et al [13]	improved YOLOv5s	Trash_ICRA19 dataset	90.6%.
Zhou et al [6]	YOLOv4	- Trashcan 1.0 - TrashCan-Instance	58.66%.
Zocco et al [4]	Improved EfficientDets	WPBB	AP: 47.4% with D0. AP: 49.6% with D3.

		TrashICRA19	- WPBB achieving AP:15%. - TrashICRA19AP: 40-50% - TrashCanAP: 10-20%.
Huang et al [7]	YOLOv5s	11,600 images	72.4%
Deng et al [32]	Mask R-CNN Data	TrashCan	44.1%
Moorton et al [8]	CNN and Logistic sigmoid function	1318 images	95%
Xue et al [5]	ResNet50-YOLOV3	3-D 10 000 images	83.4%
Sánchez-Ferrer et al [9]	Mask RCNN	CleanSea set 1223 images	66%
Zhang et al [25]	YOLOv7tCEBC, based on YOLOv7-tiny.	1570 images	84.4%
Zhu et al [26]	YOLOV8 with faster RCNN model.	Trash_ICRA19	- YOLOv8 (baseline): 3.475h with: 72.5% - YOLOv8-fasternet: 3.766h with 73.5%. - YOLOv8-C2f-Faster: 3.165h with 70.5%. - YOLOv8-C2f-Faster-EMA: 3.434h with 72.4%. - YOLOv8-C2f-Faster-EMAv2: 3.251h with 67.2%. - YOLOv8-C2f-Faster-EMAv3: 3.042h with 79.2%.
Proposed Model	Based on YOLOV10s	UW_Garbage_Debris_Dataset	YOLOV10s with 0.977h:99%

The table 17 summarizes research results that have proposed different models for underwater debris detection. The models are evaluated on various datasets, and their performance is measured using metrics i.e. Precision, Recall,

and F1-score. The existing research includes Modified U-Net, Retina Net, improved YOLOv5s, YOLOv4, Improved EfficientDets, YOLOv5s, Mask R-CNN, CNN with Logistic sigmoid function, ResNet50-YOLOV3, Mask RCNN2, YOLOv7tCEBC (based on YOLOv7-tiny), and variations of YOLOv8 with the faster RCNN model. The dataset includes MDFD and ARACATI 2017, Trash Can, Trash_ICRA19, WPBB TrashICRA19, TrashCan-Instance, a dataset of 11,600 images, TrashCan Data, 1318 images, 10,000 3-D images, CleanSea set with 1223 images, and 1570 images. Table 17 demonstrates that the existing models have low accuracy and use more time but the proposed model achieves 99% using less time than all the existing models which is 0.977h using SGD and 0.012h using AdamW.

The proposed model stands out with its impressive performance. The model achieves a remarkable accuracy of 99% while demonstrating exceptional efficiency with a training time of only 0.977 hours. This combination of high accuracy and rapid training makes it a compelling choice for underwater debris detection. Moreover, the proposed model outperforms the existing studies regarding effectiveness and efficiency.

6. Conclusions

The research explored the performance of various hyperparameters of the YOLOv10s model for underwater debris detection and examined the impact of different optimizer i.e. Adam, AdamW, and SGD with varying batch sizes and learning rates. Experiments conducted using the ICRA19 and UW_Garbage_Debris datasets demonstrated that the AdamW optimizer provided the most effective and efficient results, achieving higher precision and faster convergence with smaller batch sizes and lower learning rates. These findings confirm that a combination of a smaller batch size and lower learning rate enhances convergence stability and accelerates training. Moreover, in comparison with other YOLO variants YOLOv5, YOLOv7, and YOLOv8, the model achieved superior results validating its effectiveness for real-world underwater detection. Overall, the research successfully meets its objectives by demonstrating a high-performing, stable, and efficient single-stage detection algorithm tailored for underwater environments.

Future work will focus on expanding this system for real-time testing and deployment by integrating it into embedded hardware and conducting live inference experiments using ROV-based video feeds. Additionally, emphasis will be placed on developing advanced datasets capable of handling real-time underwater data collection challenges, along with exploring adaptive learning rates, optimizers, and modern detection frameworks to further enhance model robustness and adaptability in diverse underwater conditions.

References

- [1] L. Wei, S. Kong, Y. Wu, and J. Yu, "Image Semantic Segmentation of Underwater Garbage with Modified U-Net Architecture Model," *Sensors*, vol. 22, no. 17, pp. 1–14, 2022, doi: 10.3390/s22176546.
- [2] A. Alorf and M. U. G. Khan, "Multi-label classification of Alzheimer's disease stages from resting-state fMRI-based correlation connectivity data and deep learning," *Comput. Biol. Med.*, vol. 151, no. PA, p. 106240, 2022, doi: 10.1016/j.combiomed.2022.106240.
- [3] P. I. Lin, G. C. M. Ku, H. H. Lin, C. H. Hsu, H. C. Chi, and Y. C. Chen, "Investigating Sources of Marine Litter and Developing Coping Strategies in Scuba Diving Spots in Taiwan," *Sustain.*, vol. 14, no. 9, 2022, doi: 10.3390/su14095726.
- [4] F. Zocco, T. C. Lin, C. I. Huang, H. C. Wang, M. O. Khyam, and M. Van, "Towards More Efficient EfficientDets and Real-Time Marine Debris Detection," *IEEE Robot. Autom. Lett.*, vol. 8, no. 4, pp. 2134–2141, 2023, doi: 10.1109/LRA.2023.3245405.
- [5] B. Xue *et al.*, "An Efficient Deep-Sea Debris Detection Method Using Deep Neural Networks," *IEEE J. Sel. Top. Appl. Earth Obs. Remote Sens.*, vol. 14, pp. 12348–12360, 2021, doi: 10.1109/JSTARS.2021.3130238.
- [6] W. Zhou, F. Zheng, G. Yin, Y. Pang, and J. Yi, "YOLOTrashCan: A Deep Learning Marine Debris Detection Network," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–12, 2023, doi: 10.1109/TIM.2022.3225044.
- [7] B. Huang, G. Chen, H. Zhang, G. Hou, and M. Radenkovic, "Instant deep sea debris detection for maneuverable underwater

- machines to build sustainable ocean using deep neural network," *Sci. Total Environ.*, vol. 878, no. March, p. 162826, 2023, doi: 10.1016/j.scitotenv.2023.162826.
- [8] Z. Moorton, Z. Kurt, and W. L. Woo, "Is the use of deep learning an appropriate means to locate debris in the ocean without harming aquatic wildlife?," *Mar. Pollut. Bull.*, vol. 181, no. July, p. 113853, 2022, doi: 10.1016/j.marpolbul.2022.113853.
- [9] A. Sánchez-Ferrer, J. J. Valero-Mas, A. J. Gallego, and J. Calvo-Zaragoza, "An experimental study on marine debris location and recognition using object detection," *Pattern Recognit. Lett.*, vol. 168, pp. 154–161, 2023, doi: 10.1016/j.patrec.2022.12.019.
- [10] D. V. Politikos, E. Fakiris, A. Davvetas, I. A. Klampanos, and G. Papatheodorou, "Automatic detection of seafloor marine litter using towed camera images and deep learning," *Mar. Pollut. Bull.*, vol. 164, no. December 2020, p. 111974, 2021, doi: 10.1016/j.marpolbul.2021.111974.
- [11] D. Ma *et al.*, "MLDet: Towards efficient and accurate deep learning method for Marine Litter Detection," *Ocean Coast. Manag.*, vol. 243, no. March, 2023, doi: 10.1016/j.ocecoaman.2023.106765.
- [12] K. Cheng, L. Yan, Y. Ding, H. Zhou, M. Li, and H. Abdul Ghafoor, "Sonar image garbage detection via global despeckling and dynamic attention graph optimization," *Neurocomputing*, vol. 529, pp. 152–165, 2023, doi: 10.1016/j.neucom.2023.01.081.
- [13] X. Teng, Y. Fei, K. He, and L. Lu, "The Object Detection of Underwater Garbage with an Improved YOLOv5 Algorithm," *ACM Int. Conf. Proceeding Ser.*, pp. 55–60, 2022, doi: 10.1145/3549179.3549189.
- [14] X. Zhang, D. Zhu, and W. Gan, "YOLOv7t-CEBC Network for Underwater Litter Detection," *J. Mar. Sci. Eng.*, vol. 12, no. 4, 2024, doi: 10.3390/jmse12040524.
- [15] J. Zhu, T. Hu, L. Zheng, N. Zhou, H. Ge, and Z. Hong, "YOLOv8-C2f-Faster-EMA: An Improved Underwater Trash Detection Model Based on YOLOv8," *Sensors*, vol. 24, no. 8, 2024, doi: 10.3390/s24082483.
- [16] G. Wang, Y. Chen, P. An, H. Hong, J. Hu, and T. Huang, "UAV-YOLOv8: A Small-Object-Detection Model Based on Improved YOLOv8 for UAV Aerial Photography Scenarios," *Sensors*, vol. 23, no. 16, 2023, doi: 10.3390/s23167190.
- [17] U. Sirisha, S. P. Praveen, P. N. Srinivasu, P. Barsocchi, and A. K. Bhoi, "Statistical Analysis of Design Aspects of Various YOLO-Based Deep Learning Models for Object Detection," *Int. J. Comput. Intell. Syst.*, vol. 16, no. 1, pp. 1–29, 2023, doi: 10.1007/s44196-023-00302-w.
- [18] E. Lieskovsk and B. Buř, "applied sciences Comparison of CNN-Based Models for Pothole Detection in Real-World Adverse Conditions : Overview and Evaluation," 2023.
- [19] M. Hussain, "YOLO-v1 to YOLO-v8, the Rise of YOLO and Its Complementary Nature toward Digital Manufacturing and Industrial Defect Detection," *Machines*, vol. 11, no. 7, 2023, doi: 10.3390/machines11070677.
- [20] T. Diwan, G. Anirudh, and J. V. Tembhurne, "Object detection using YOLO: challenges, architectural successors, datasets and applications," *Multimed. Tools Appl.*, vol. 82, no. 6, pp. 9243–9275, 2023, doi: 10.1007/s11042-022-13644-y.
- [21] P. Jiang, D. Ergu, F. Liu, Y. Cai, and B. Ma, "A Review of Yolo Algorithm Developments," *Procedia Comput. Sci.*, vol. 199, pp. 1066–1073, 2021, doi: 10.1016/j.procs.2022.01.135.
- [22] F. M. Talaat and H. ZainEldin, "An improved fire detection approach based on YOLO-v8 for smart cities," *Neural Comput.*

Appl., vol. 35, no. 28, pp. 20939–20954, 2023, doi: 10.1007/s00521-023-08809-1.

- [23] S. Lee and T. Kim, "Impact of Deep Learning Optimizers and Hyperparameter Tuning on the Performance of Bearing Fault Diagnosis," *IEEE Access*, vol. 11, no. May, pp. 55046–55070, 2023, doi: 10.1109/ACCESS.2023.3281910.
- [24] H. Naz *et al.*, "Ensembled Deep Convolutional Generative Adversarial Network for Grading Imbalanced Diabetic Retinopathy Recognition," *IEEE Access*, vol. 11, no. October, pp. 120554–120568, 2023, doi: 10.1109/ACCESS.2023.3327900.
- [25] A. Intasam *et al.*, "Optimizing the Hyperparameter Tuning of YOLOv5 for Breast Cancer Detection," *2023 9th Int. Conf. Eng. Appl. Sci. Technol. ICEAST 2023 - Proceeding*, pp. 184–187, 2023, doi: 10.1109/ICEAST58324.2023.10157611.
- [26] S. Sriram, P. Aburvan, T. P. Arun Kaarthic, N. Vijayaraj, and T. Murugan, "Enhanced YOLOv10 Framework Featuring DPAM and DALSM for Real-Time Underwater Object Detection," *IEEE Access*, vol. 13, no. December 2024, pp. 8691–8708, 2025, doi: 10.1109/ACCESS.2025.3527315.
- [27] A. Ghahremani, S. D. Adams, M. Norton, S. Y. Khoo, and A. Z. Kouzani, "Detecting Defects in Solar Panels Using the YOLO v10 and v11 Algorithms," *Electron.*, vol. 14, no. 2, 2025, doi: 10.3390/electronics14020344.
- [28] N. B. A. Karna, M. A. P. Putra, S. M. Rachmawati, M. Abisado, and G. A. Sampedro, "Toward Accurate Fused Deposition Modeling 3D Printer Fault Detection Using Improved YOLOv8 With Hyperparameter Optimization," *IEEE Access*, vol. 11, no. July, pp. 74251–74262, 2023, doi: 10.1109/ACCESS.2023.3293056.
- [29] R.-Y. Ju and W. Cai, "Fracture detection in pediatric wrist trauma X-ray images using YOLOv8 algorithm," *Sci. Rep.*, vol. 13, no. 1, pp. 1–13, 2023, doi: 10.1038/s41598-023-47460-7.
- [30] E. Soylu and T. Soylu, *A performance comparison of YOLOv8 models for traffic sign detection in the Robotaxi-full scale autonomous vehicle competition*, no. 0123456789. Springer US, 2023. doi: 10.1007/s11042-023-16451-1.
- [31] I. S. Isa, M. S. A. Rosli, U. K. Yusof, M. I. F. Maruzuki, and S. N. Sulaiman, "Optimizing the Hyperparameter Tuning of YOLOv5 for Underwater Detection," *IEEE Access*, vol. 10, pp. 52818–52831, 2022, doi: 10.1109/ACCESS.2022.3174583.
- [32] H. Deng, D. Ergu, F. Liu, B. Ma, and Y. Cai, "An embeddable algorithm for automatic garbage detection based on complex marine environment," *Sensors*, vol. 21, no. 19, 2021, doi: 10.3390/s21196391.

Conflict of interest

There is no conflict interest.

Journal Pre-proof